

LeetMastery By Pattern Python Only · Print Edition · 9-up Portrait 331 questions · 21 patterns · bold black · 9-up portrait Descriptions + images · Brute force · All 4 sites Sorted fewest→most	Contents		#1 Graphs (5) M3 H2		#383 Random Node (Easy) 734 Sentence Similarity (Easy) #1405 Single Row Keyboard (Easy) #8 String to Integer (atoi) (Medium) 4249 Group Shifted Strings (Medium) 4271 Encode and Decode Strings (Medium) 6833 Design Log Storage System (Medium) #1118 Alphabet Board Path (Hard)		#5 Sliding Window (9) M8 H1		#9 BFS (10) M6 H4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#4 Sliding Window (9) M8 H1		#1122 Relative Sort Array (Easy) 449 Group Anagrams (Medium) 456 Merge Intervals (Medium) #1152 Analyze User Website Visit Pattern (Medium)		#7 Bit Manipulation (10) E6 M4
---	--	--	--	--	--	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	--

Sorting	p.319	Sorting	p.320	Sorting	Sheet 9/86 · mini-pages 289-324	Sorting	Sheet Mastery 36-up	p.322	Sorting	p.323	Sorting	p.324
---------	-------	---------	-------	---------	---------------------------------	---------	---------------------	-------	---------	-------	---------	-------

EAS

Reporting

Reporting

• LeafDrops

- **LeetDoocs**

Reporting

Reporting

Reporting

Reporting

portion

Reporting

Reporting

Community Solutions (Python)

Sheet 10/86 · mini-page

Reporting

DringLeetMastery 36-up

Sorting

Starting

Starting

#1122 Relative Sort Array

Starting

Starting

they appeared in the pattern.

exist in a single pass.

The solution uses a backtracking approach. For each empty cell on the board, we attempt to place a digit from '1' to '9'. Before placing a digit, we check if the placement is valid by ensuring that: - The digit is not already present in the same row. - The digit is not already present in the same column. - The digit is not already present in the corresponding 3x3 sub-box. If a valid digit is found, we place it on the board and recursively attempt to solve the rest of the board. If no valid digit can be placed, the algorithm backtracks by reverting the last move and trying a different digit. This recursive search continues until the board is completely filled with a valid Sudoku configuration. #51 N-Queens [Hard] Key Insights • Use backtracking to explore potential positions row by row. • Use sets to track which columns and diagonals are under attack. • Diagonals can be tracked by (row - col) and (row + col), ensuring that queens do not conflict along any diagonal. • Recursively build each valid board configuration and backtrack when a conflict arises. Space & Time	Time Complexity: $O(n!)$ in the worst-case scenario, as the solution explores permutations for queen placements. Space Complexity: $O(n^2)$ for storing board configurations and $O(n)$ for recursion stack and conflict-tracking sets. Solution We use a backtracking algorithm that places queens row by row. For each row, we try placing a queen in each column. Before placing, we check if the column, main diagonal (row - col), and anti-diagonal (row + col) are free using dedicated sets. If a queen can be safely placed, we mark the column, diagonals as occupied and recurse to the next row. Once all rows are processed, the board configuration is a valid solution and is added to the result. After that, we backtrack by unmarking the sets and removing the queen from the board. This systematic approach ensures all possible configurations are considered. #126 Word Ladder II [Hard] Key Insights • Use Breadth-First Search (BFS) to traverse level-by-level and capture only the shortest paths. • Build a parent mapping during BFS to record which words lead to each newly discovered word.	• Once the shortest path is found via BFS, use backtracking (or DFS) from endWord to reconstruct all transformation sequences. • Remove words as they are visited within a level to avoid unnecessary revisits and cycles. • The solution must handle multiple shortest paths, so maintaining a list of predecessors for each word is crucial. Space & Time Time Complexity: $O(N \cdot L^2 \cdot 26^L)$ where N is the number of words in the wordlist and L is the length of each word. This accounts for checking all possible one-letter transformations. Space Complexity: $O(N \cdot L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures. Solution We solve the problem in two major phases: - Use BFS to explore from beginWord to endWord. During BFS, record all parents for each word encountered. This ensures we capture all possible predecessors that yield a shortest transformation. - Once the BFS completes (once endWord is reached), use backtracking (DFS) beginning from the endWord and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from	beginWord to endWord. This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration. #996 Number of Squareful Arrays [Hard] Key Insights • The problem requires ensuring adjacent pairs sum up to a perfect square. • Sorting the array helps in efficiently handling duplicates; when iterating in the backtracking, skip duplicate elements to avoid overcounting. • For each valid candidate, check that the sum with the previous number (if any) is a perfect square. • Use backtracking (DFS) to generate all valid permutations while maintaining a visited flag for each index. • Since the array may contain duplicates, extra care is taken to only use each occurrence in a controlled order (i.e., if two identical numbers exist, only the first unvisited instance can be chosen at a given recursive call). Space & Time Time Complexity: $O(2^n \cdot 2^n)$ in the worst-case scenario due to exploring all bitmask states with n elements and checking pairwise sums.	Space Complexity: $O(n)$ for recursion and $O(n)$ for the visited flag, plus potentially $O(2^n \cdot n)$ for memoization if implemented. Solution The solution uses a backtracking algorithm to explore all permutations. The array is first sorted to handle duplicates. A helper function checks if a number is a perfect square and is used to validate that the sum of every adjacent pair is a perfect square. During DFS, a visited array is maintained to track elements included so far. Skipping duplicate elements (unless the earlier duplicate was used) is essential to avoid overcounting. This approach ensures that every valid permutation is considered exactly once. BFS -1 A wall or an obstacle. 0 A gate. - INF Infinity means an empty room. We use the value 231 - 1 = 2147483647 to represent INF as you may assume that the distance to a gate is less than 2147483647. Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF. Example 1:	BFS #286 Walls and Gates LeetCode - SimplifyLeet - WalkCC - LeetCode Array - BFS - Matrix Lists - Premium 58 Time Complexity: $O(m \cdot n)$ where m and n are the dimensions of the grid, as each cell is processed at most once. Space Complexity: $O(m \cdot n)$ in the worst-case scenario used by the BFS queue. Problem You are given an $m \times n$ grid rooms initialized with these three possible values: -1 A wall or an obstacle. 0 A gate. - INF Infinity means an empty room. We use the value 231 - 1 = 2147483647 to represent INF as you may assume that the distance to a gate is less than 2147483647. Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF. Example 1:
Backtracking #541	Backtracking #542	Backtracking #543	Backtracking #544	Backtracking #545	BFS #546
BFS #547	BFS #548	BFS #549	BFS #550	BFS #551	BFS #552
BFS #553	BFS #554	BFS #555	BFS #556	BFS #557	BFS #558
BFS #559	BFS #560	BFS #561	BFS #562	BFS #563	BFS #564
BFS #565	BFS #566	BFS #567	BFS #568	BFS #569	BFS #570
BFS #571	BFS #572	BFS #573	BFS #574	BFS #575	BFS #576

[illegible]

Python solution for Missing Ranges Problem <pre>def findMissingRanges(nums, lower, upper): missing_ranges = [] # List to store the missing range intervals prev = lower - 1 # Initialize previous value to one less than lower # Append a sentinel at the end for easier gap handling nums.append(upper + 1) # Loop through each number, including the sentinel for num in nums: # If there is a gap between previous and current number if num - prev > 2: # If gap is of single number, add that number as a range if num - prev == 2: missing_ranges.append([prev + 1, prev + 1]) else: missing_ranges.append([prev + 1, num - 1]) prev = num # Update previous to current return missing_ranges</pre> Arrays & Hashing # Python solution for Missing Ranges Problem # Example usage: print(findMissingRanges([0,1,3,50,75], 0, 99)) • LeetCode class Solution: def findMissingRanges(self, nums: List[int], lower: int, upper: int) -> List[List[int]]: n = len(nums) if n == 0: return [[lower, upper]] ans = [] if nums[0] > lower: ans.append([lower, nums[0] - 1]) for a, b in pairwise(nums): if b - a > 1: ans.append([a + 1, b - 1]) if nums[-1] < upper: ans.append([nums[-1] + 1, upper]) return ans • LeetCode Input ["MissingRange", "mat", "mat", "mat", "mat"] [[3], [1], [10], [3], [5]] Output [[mat], [1, 0, 5, 5, 4, 6, 6, 6, 7, 8]] Explanation MovingAverage movingRange = new MovingAverage(3);	Python solution for Moving Average from Data Stream <pre>from collections import deque # Import deque for efficient queue operations class MovingAverage: def __init__(self, size): # Initialize with the maximum window size self.size = size self.queue = deque() # Queue to store current window values self.current_sum = 0 # Running sum of the current window def next(self, val): # Add new value to the window if len(self.queue) == self.size: # If the window is full, remove the oldest element and update the sum removed_val = self.queue.popleft() self.current_sum -= removed_val # Subtract its value from the running sum # Append the new value to the queue and update the sum self.queue.append(val) self.current_sum += val</pre> Arrays & Hashing # Python solution for Moving Average from Data Stream # Calculate and return the moving average by dividing the running sum by the current window size return self.current_sum / len(self.queue) • LeetCode class MovingAverage: def __init__(self, size: int): self.arr = [] self.size = size self.cnt = 0 def next(self, val: int) -> float: idx = self.cnt - len(self.arr) self.cnt += 1 self.arr[idx] = val array self.s = val + len(self.arr[idx]) self.arr[idx] = val self.cnt += 1 return self.s / min(self.cnt, self.size) • LeetCode Input ["MovingAverage", "next", "mat", "mat", "mat"] [[3], [1], [10], [3], [5]] Output [[mat], [1, 0, 5, 5, 4, 6, 6, 6, 7, 8]] Explanation MovingAverage movingRange = new MovingAverage(3);	Arrays & Hashing #346 Moving Average from Data Stream LeetCode - SimpleKey - WalKCC - LeetCode Array - Design - Queue - Data Stream Lists: Premium 58 Time Complexity: O(1) per call to next() Space Complexity: O(window size) Problem Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window. Implement the MovingAverage class: MovingAverage(int size) Initializes the object with the size of the window size. double next(int val) Returns the moving average of the last size values of the stream. Example 1: Input ["MovingAverage", "next", "mat", "mat", "mat"] [[3], [1], [10], [3], [5]] Output [[mat], [1, 0, 5, 5, 4, 6, 6, 6, 7, 8]] Explanation MovingAverage movingRange = new MovingAverage(3); Arrays & Hashing #346 Moving Average from Data Stream LeetCode - SimpleKey - WalKCC - LeetCode Array - Design - Queue - Data Stream Lists: Premium 58 Time Complexity: O(1) per call to next() Space Complexity: O(window size)	Arrays & Hashing Constraints: 1 <= size <= 1000 -105 <= val <= 105 At most 104 calls will be made to next. Community Solutions (Python) • WalKCC class MovingAverage: def __init__(self, size: int): self.size = size self.sum = 0 self.q = collections.deque() def next(self, val: int) -> float: if len(self.q) == self.size: self.sum -= self.q.popleft() self.sum += val self.q.append(val) return self.sum / len(self.q) • LeetCode Arrays & Hashing class MovingAverage: def __init__(self, size: int): self.s = 0 self.data = [] self.cnt = 0 def next(self, val: int) -> float: i = self.cnt % len(self.data) self.s += val - self.data[i] self.data[i] = val self.cnt += 1 return self.s / min(self.cnt, len(self.data)) # Your MovingAverage object will be instantiated and called as such: # obj = MovingAverage(size) # param_1 = obj.next(val)	Arrays & Hashing #760 Find Anagram Mappings LeetCode - SimpleKey - WalKCC - LeetCode Array - Hash Table Lists: Premium 58 Time Complexity: O(n) where n is the number of elements in arrays (since there is bound to be the dictionary and another pass to find the mappings). Space Complexity: O(n) for storing the dictionary that Problem You are given two integer arrays nums1 and nums2 where nums2 is an anagram of nums1. Both arrays may contain duplicates. Return an index mapping array mapping from nums1 to nums2 where mapping[i] = j means the ith element in nums1 appears in nums2 at index j. If there are multiple answers, return any of them. An array a is an anagram of an array b means b is made by randomizing the order of the elements in a. Example 1: Arrays & Hashing #760 Find Anagram Mappings LeetCode - SimpleKey - WalKCC - LeetCode Array - Hash Table Lists: Premium 58 Time Complexity: O(n) where n is the number of elements in arrays (since there is bound to be the dictionary and another pass to find the mappings). Space Complexity: O(n) for storing the dictionary that	Arrays & Hashing #760 Find Anagram Mappings LeetCode - SimpleKey - WalKCC - LeetCode Array - Hash Table Lists: Premium 58 Time Complexity: O(n) where n is the number of elements in arrays (since there is bound to be the dictionary and another pass to find the mappings). Space Complexity: O(n) for storing the dictionary that Problem You are given two integer arrays nums1 and nums2 where nums2 is an anagram of nums1. Both arrays may contain duplicates. Return an index mapping array mapping from nums1 to nums2 where mapping[i] = j means the ith element in nums1 appears in nums2 at index j. If there are multiple answers, return any of them. An array a is an anagram of an array b means b is made by randomizing the order of the elements in a. Example 1: Arrays & Hashing #760 Find Anagram Mappings LeetCode - SimpleKey - WalKCC - LeetCode Array - Hash Table Lists: Premium 58 Time Complexity: O(n) where n is the number of elements in arrays (since there is bound to be the dictionary and another pass to find the mappings). Space Complexity: O(n) for storing the dictionary that
Python solution for Moving Average from Data Stream <pre>from collections import deque # Import deque for efficient queue operations class MovingAverage: def __init__(self, size): # Initialize with the maximum window size self.size = size self.queue = deque() # Queue to store current window values self.current_sum = 0 # Running sum of the current window def next(self, val): # Add new value to the window if len(self.queue) == self.size: # If the window is full, remove the oldest element and update the sum removed_val = self.queue.popleft() self.current_sum -= removed_val # Subtract its value from the running sum # Append the new value to the queue and update the sum self.queue.append(val) self.current_sum += val</pre> Arrays & Hashing # Python solution for Moving Average from Data Stream # Calculate and return the moving average by dividing the running sum by the current window size return self.current_sum / len(self.queue) • LeetCode class MovingAverage: def __init__(self, size: int): self.arr = [] self.size = size self.cnt = 0 def next(self, val: int) -> float: idx = self.cnt - len(self.arr) self.cnt += 1 self.arr[idx] = val array self.s = val + len(self.arr[idx]) self.arr[idx] = val self.cnt += 1 return self.s / min(self.cnt, self.size) • LeetCode Input ["MovingAverage", "next", "mat", "mat", "mat"] [[3], [1], [10], [3], [5]] Output [[mat], [1, 0, 5, 5, 4, 6, 6, 6, 7, 8]] Explanation MovingAverage movingRange = new MovingAverage(3);	Arrays & Hashing #760 Find Anagram Mappings				

Arrays & Hashing	p.937	Arrays & Hashing	p.938	Arrays & Hashing	p.939	Arrays & Hashing	p.940	Arrays & Hashing	p.941	Arrays & Hashing	p.942
------------------	-------	------------------	-------	------------------	-------	------------------	-------	------------------	-------	------------------	-------

Aktuwa w roztworze	p.242	Aktuwa w roztworze	p.244	Aktuwa w roztworze	p.242	Aktuwa w roztworze	p.240	Aktuwa w roztworze	p.242	Aktuwa w roztworze	p.240
--------------------	-------	--------------------	-------	--------------------	-------	--------------------	-------	--------------------	-------	--------------------	-------

[illegible]

2. Calculate the sum of the elements of nums between min(nums[0], nums[-1]) and max(nums[0], nums[-1]).

called as such:	current tree node	return s	called as such:	return None
-----------------	-------------------	----------	-----------------	-------------

[illegible]

called as such:

p.1279	Heap	p.1280	Heap	p.1281	Heap	p.1282	Heap	p.1283	Heap
<pre> class Solution: def mergeKLists(self, lists: List(ListNode)) -> ListNode: if n == len(lists): return None for i in range(n - 1): # This is O(N) times of merge, while if we used divide-and-conquer then it's o(logN) times of merge. lists[i] += 1 self.mergeKLists(lists[1:], lists[i + 1]) return lists[0] def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode: dummy = ListNode(0) cur = dummy while l1 and l2: if l1.val <= l2.val: cur.next = l1 l1 = l1.next else: cur.next = l2 l2 = l2.next cur = cur.next </pre>	<pre> cur.next = l1 or l2 return dummy.next if __name__ == '__main__': l1 = ListNode(1) l1.next = ListNode(4) l2.next = ListNode(3) l2.next = ListNode(5) l2 = ListNode(1) l2.next = ListNode(3) l2.next = ListNode(4) l3 = ListNode(2) l3.next = ListNode(6) print(Solution().mergeKLists([l1, l2, l3])) print(Solution().mergeKLists([[]])) print(Solution().mergeKLists([[]])) </pre>	#128 The Skyline Problem LeetCode - Simplify LC - LeetCode Array - Divide and Conquer - Binary Indexed Tree - Segment Tree - Line Sweep - Heap - Ordered Set Louis Deeny Zhang Time Complexity: O(n log n) due to sorting events and heap operations for each building event. Space Complexity: O(n) for the event list and the heap used for processing active buildings. Problem A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively. The geometric information of each building is given in the array buildings where buildings[i] = [lefti, righti, heighti]: • lefti is the x coordinate of the left edge of the ith building. • righti is the x coordinate of the right edge of the ith building. • heighti is the height of the ith building.	You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0. The skyline should be represented as a list of "key points" sorted by their x-coordinate in the form [x1,y1,x2,y2,...]. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has y==coordinate 0 and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour. Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, [[-1,2],[1,2],[3,2],[4,2]] is not acceptable; the three lines of height 2 should be merged into one in the final output as such: [[-1,2],[4,2]]. Example 1:	<pre> Input: buildings = [[1,2,3], [2,7,15], [5,12,12], [15,20,18], [19,24,8]] Output: [[1,2,3], [2,7,15], [7,12,12], [12,8], [15,18], [20,18], [24,0]] Explanation: Figure A shows the buildings of the input. Figure B shows the skyline formed by those buildings. The red points in Figure B represent the key points in the output list. Example 2: Input: buildings = [[0,2,3],[2,5,3]] Output: [[0,3],[2,3],[5,0]] Constraints: n == buildings.length 0 <= lefti < righti <= 10^4 1 <= heighti <= 231 - 1 buildings is sorted by lefti in non-decreasing order. </pre> Brute Force (Python)	Brute Force Approach class Solution: <pre> def getKeyLine(self, buildings): if Brute Force O(n Log n) - sort repeatedly instead of heap data = List(buildings) result = [] for i in range(len(data)): result.append((data.pop(0))) return result </pre> Community Solutions (Python) • Heuristic <pre> class Solution: def getKeyLine(self, buildings: List[List(int)]) -> List[List(int]): n = len(buildings) if n == 0: return [] if n == 1: left, right, height = buildings[0] return [(left, height), (right, 0)] left = self.getKeyLine(buildings[:n // 2]) </pre>				

Heap	p.1291	Heap	p.1292	Heap	Sheet 36/86 · mini-pages 42/61 · 1296 · LeetMastery 36-up	p.1294	Heap	p.1295	Heap	p.1296
------	--------	------	--------	------	---	--------	------	--------	------	--------

Head	0.1333	Head	0.1334	Head	0.1335	Head	0.1336	Head	0.1337	Head	0.1338
------	--------	------	--------	------	--------	------	--------	------	--------	------	--------

Мазо	п. 1220	Мазо	п. 1240	Мазо	п. 1241	Мазо	п. 1242	Мазо	п. 1242	Мазо	п. 1244
------	---------	------	---------	------	---------	------	---------	------	---------	------	---------

Hann	n = 3347	Hann	n = 3346	Hann	n = 3347	Hann	n = 3346	Hann	n = 3346	Hann	n = 3346
------	----------	------	----------	------	----------	------	----------	------	----------	------	----------

[illegible][illegible][illegible]

Community Solutions (Python)

• WalKCC

```

class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        rows = len(matrix)
        cols = len(matrix[0])
        for i in range(rows - 1):
            for j in range(len(matrix[0]) - 1):
                if matrix[i][j] == matrix[i + 1][j + 1]:
                    return True
            return False
        return True

```

• LeetCode

```

class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        m, n = len(matrix), len(matrix[0])
        for i in range(m, 0):
            for j in range(n, 0):
                if matrix[i][j] == matrix[i - 1][j - 1]:
                    return True
            return False
        return True

```

• SimpliLeet

class Solution:

```

def isToeplitzMatrix(matrix):
    List(list(int)) -> bool:
        n = len(matrix), len(matrix[0])
        return all(
            matrix[i][j] == matrix[i - 1][j - 1]
            for i in range(1, n)
            for j in range(1, m)
        )

```

Matrix

#867 Transpose Matrix

LeetCode - SimpliLeet - WalKCC - LeetCode

Array - Simulation

Time Complexity: $O(m \times n)$ Space Complexity: $O(m \times n)$ for the new matrix holding the transposed values

Problem

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

1	4	5
-10	5	16
10	-7	6

→

1	-10	10
4	5	-7
5	16	6

Example 1:

```

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[1,4,7],[2,5,8],[3,6,9]]

```

Example 2:

```

Input: matrix = [[1,2,3],[4,5,6]]
Output: [[1,4],[2,5],[3,6]]

```

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix[0].length}$
- $1 \leq m, n \leq 1000$
- $1 \leq m \times n \leq 10^5$
- $-10^9 \leq \text{matrix[i][j]} \leq 10^9$

Brute Force (Python)

```

# Brute Force Approach
class Solution:
    def transpose(self, matrix):
        ans = [[0] * len(A) for _ in range(len(A[0]))]
        for i in range(len(A)):
            for j in range(len(A[0])):
                ans[j][i] = A[i][j]
        return ans

```

• LeetCode

```

class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        ans = [[0] * len(A) for _ in range(len(A[0]))]
        for i in range(len(A)):
            for j in range(len(A[0])):
                ans[j][i] = A[i][j]
        return ans

```

• SimpliLeet

p.1405	p.1406	p.1407	p.1408	p.1409	p.1410
# Python solution with clear comments <pre>def transpose(matrix): # Number of rows in the original matrix num_rows = len(matrix) # Number of columns in the original matrix num_cols = len(matrix[0]) # Create a new matrix with dimensions swapped: num_cols x num_rows transposed = [[0] * num_rows for _ in range(num_cols)] # Iterate through each element in the original # matrix for i in range(num_rows): for j in range(num_cols): # Place the element at the swapped # index in the transposed matrix transposed[j][i] = matrix[i][j] return transposed</pre> # Example usage: <pre>matrix = [[1, 2, 3], [4, 5, 6]] print(transpose(matrix))</pre> • LC.ca	class Solution: <pre>def transpose(self, matrix: List[List[int]]) -> List[List[int]]: return List(zip(*matrix))</pre>	Matrix #2373 Largest Local Values in a Matrix LeetCode · SimplyList · WalkCC · LeetCode Array · Matrix Signal · CodeSignal Time Complexity: $O(n^2 \times 2)$ - We iterate over $(n-2) \times (n-2)$ positions, and for each position, we look at 9 elements. Space Complexity: $O(n \times 2)$ - The additional space is required for the output matrix. Problem You are given an $n \times n$ integer matrix grid. Generate an integer matrix <code>maxLocal</code> of size $(n - 2) \times (n - 2)$ such that: <code>maxLocal[i][j]</code> is equal to the largest value of the 3×3 matrix in grid centered around <code>row = i + 1</code> and <code>column = j + 1</code>. In other words, we want to find the largest value in every contiguous 3×3 matrix in grid. Return the generated matrix. Example 1:	Input: grid = [[9,9,8,1],[5,6,4,3],[8,2,6,4],[6,2,2,2]] Output: [[9,9],[8,6]] Explanation: The diagram shows above the original matrix and the generated matrix. Notice that each value in the generated matrix corresponds to the largest value of a contiguous 3×3 matrix in grid. Example 2:	Input: grid = [[1,1,1,1,1],[1,1,1,1,1],[1,1,1,1,1],[1,1,1,1,1],[1,1,1,1,1]] Output: [[2,2,2],[2,2,2],[2,2,2]] Explanation: Notice that the 2 is contained within every contiguous 3×3 matrix in grid. Constraints: <code>n == grid.length == grid[0].length</code> <code>3 <= n <= 100</code> <code>1 <= grid[i][j] <= 100</code> Brute Force (Python)	# Brute Force Approach class Solution: <pre>def largestLocal(self, grid): # Brute Force O(n^2 * n^2) - flood fill from # every cell n = len(grid) result = [] for r in range(n): for c in range(n): visited = set() def floodFill(r, c): if (r, c) in visited or not (isRowIn and isColIn): return visited.add((r, c)) for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]: floodFill(r+dr, c+dc) result.append(max(result, len(visited))) return result</pre> Community Solutions (Python) • WalkCC

p.1411	p.1412	p.1413	p.1414	p.1415																																																																																
<pre> class Solution: def largestLocal(self, grid: List[List(int)]) -> List[List(int)]: n = len(grid) ans = [[0] * (n - 2) for _ in range(n - 2)] for i in range(n - 2): for j in range(n - 2): for k in range(1, i + 3): for l in range(1, j + 3): ans[i][j] = max(ans[i][j], grid[x][y]) return ans # LeetCode class Solution: def largestLocal(self, grid: List[List(int)]) -> List[List(int)]: n = len(grid) ans = [[0] * (n - 2) for _ in range(n - 2)] for i in range(n - 2): for j in range(n - 2): for k in range(1, i + 3): for l in range(1, j + 3): ans[i][j] = max(ans[i][j], grid[x][y]) return ans </pre> • Simply Local	<pre> # Python solution for finding the largest local values in a matrix def largestLocal(grid): n = len(grid) # dimension of the grid # Initialize the result matrix with zeros, of size (n-2) x (n-2) result = [[0] * (n - 2) for _ in range(n - 2)] # Loop through each possible top-left index of a 3x3 submatrix for i in range(n - 2): for j in range(n - 2): # Compute the maximum in the current 3x3 submatrix local_max = 0 # since grid elements are >= 1, starting with 0 is safe for k in range(3): local_max = max(local_max, grid[i + k][j + l]) # Store the local maximum in the result matrix result[i][j] = local_max return result # Example usage: </pre>	<pre> grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]] print(largestLocal(grid)) # LeetCode class Solution: def largestLocal(self, grid: List[List(int)]) -> List[List(int)]: n = len(grid) ans = [[0] * (n - 2) for _ in range(n - 2)] for i in range(n - 2): for j in range(n - 2): ans[i][j] = max(grid[x][y] for x in range(i, i + 3) for y in range(j, j + 3)) return ans </pre>	Matrix #36 Valid Sudoku LeetCode - Simply Local - WalkCC - LeetCode Array - Hash Table - Matrix Lines Grid 109 Time Complexity: O(1) – Since the board size is fixed at 9x9. Space Complexity: O(1) – The extra space used by the sets is bounded by the fixed board size. Problem Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules: - Each row must contain the digits 1-9 without repetition. - Each column must contain the digits 1-9 without repetition. - Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without repetition. Note: - A Sudoku board (partially filled) could be valid but is not necessarily solvable. - Only the filled cells need to be validated according to the mentioned rules. Example 1: <table border="1" data-bbox="1136 730 1331 783"> <tr><td>5</td><td>3</td><td></td><td>7</td><td></td><td></td><td></td><td></td><td></td></tr> <tr><td>6</td><td></td><td></td><td>1</td><td>9</td><td>5</td><td></td><td></td><td></td></tr> <tr><td></td><td>9</td><td>8</td><td></td><td></td><td></td><td>6</td><td></td><td></td></tr> <tr><td>8</td><td></td><td></td><td>6</td><td></td><td></td><td></td><td></td><td>3</td></tr> <tr><td>4</td><td></td><td>8</td><td></td><td>3</td><td>1</td><td></td><td></td><td></td></tr> <tr><td>7</td><td></td><td></td><td>2</td><td></td><td></td><td></td><td>6</td><td></td></tr> <tr><td></td><td>6</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr> <tr><td></td><td></td><td>4</td><td>1</td><td>9</td><td></td><td></td><td>5</td><td></td></tr> <tr><td></td><td></td><td></td><td>8</td><td></td><td></td><td>7</td><td>9</td><td></td></tr> </table> Input: board = <pre> [["5","3"," ","7"," "," "," "," "," "], ["6"," "," ","1","9","5"," "," "," "], [" ","9","8"," "," "," ","6"," "," "], ["8"," "," ","6"," "," "," "," ","3"], ["4"," ","8"," ","3","1"," "," "," "], ["7"," "," ","2"," "," "," "," ","6"], [" ","6"," "," "," "," "," "," "," "], [" "," ","4","1","9"," "," ","5"," "], [" "," "," ","8"," "," ","7","9"," "]] </pre> Constraints: - boardLength == 9 - board[0].length == 9 - board[i][j] is a digit 1-9 or ".". Brute Force (Python) # Brute Force Approach class Solution: <pre> def isValidSudoku(self, board): # Brute Force O(1) – check each row, col, box for duplicates row = [set()] for i in range(9) col = [set()] for i in range(9) boxes = [set()] for i in range(9) for i in range(9): for j in range(9): val = board[i][j] </pre>	5	3		7						6			1	9	5					9	8				6			8			6					3	4		8		3	1				7			2				6			6										4	1	9			5					8			7	9	
5	3		7																																																																																	
6			1	9	5																																																																															
	9	8				6																																																																														
8			6					3																																																																												
4		8		3	1																																																																															
7			2				6																																																																													
	6																																																																																			
		4	1	9			5																																																																													
			8			7	9																																																																													

Matrix	P.L427	Matrix	P.L428	Matrix	P.L429	Matrix	P.L430	Matrix	P.L431	Matrix	P.L432
	<pre> if val == '.': continue box_idx = (r // 3) * 3 + c // 3 if val in row[r] or val in col[c] or val in boxes[box_idx]: return False seen.add(c + 'row' + str(i)) seen.add(c + 'col' + str(j)) seen.add(c + 'box' + str(i // 3) + str(j // 3)) return True </pre>	<pre> c = 'col' + str(i) in seen or c + 'box' + str(i // 3) + str(j // 3) in seen: return False seen.add(c + 'row' + str(i)) seen.add(c + 'col' + str(j)) seen.add(c + 'box' + str(i // 3) + str(j // 3)) return True </pre>	<pre> num = int(c) - 1 k = i // 3 * 3 + j // 3 if row[i][num] or col[j][num] or sub[k][num]: return False row[i][num] = True sub[k][num] = True return True </pre>	<pre> num = board[r][c] # Skip empty cells if num == '.': continue # Compute box index using integer division box_index = (r // 3) * 3 + (c // 3) # Check if the number is already present in the row, column, or box if num in row[r] or num in col[c] or num in boxes[box_index]: return False # Add number to row, column, and box sets row[r].add(num) col[c].add(num) boxes[box_index].add(num) return True # Example usage: if __name__ == '__main__': board = [['5', '3', '4', '6', '7', '8', '9', '2', '1', '0'], </pre>	<pre> ["6","9","5","1","7","8","2","3","4","0"], ["3","8","9","0","6","7","5","2","4","1"], ["4","2","0","3","8","9","7","6","5","1"], ["7","6","0","4","1","5","8","9","3","2"], ["9","6","0","2","8","7","5","3","4","1"], ["8","7","1","9","4","3","0","6","5","2"], ["2","1","0","8","6","3","9","7","5","4"],] print(isValidSudoku(board)) # Expected output: True </pre>	<pre> num = int(c) - 1 k = i // 3 * 3 + j // 3 if row[i][num] or col[j][num] or sub[k][num]: return False row[i][num] = True sub[k][num] = True return True </pre>					
Community Solutions (Python)											
+ WalkCC											
<pre> class Solution: def isValidSudoku(self, board: List[List[str]]) -> bool: seen = set() for i in range(9): for j in range(9): c = board[i][j] if c == '.': continue if (c + 'row' + str(i) in seen or </pre>	<pre> class Solution: def isValidSudoku(self, board: List[List[str]]) -> bool: seen = {} col = {} row = {} for i in range(9): for j in range(9): c = board[i][j] if c == '.': continue </pre>	<pre> # Python solution with inline comments def isValidSudoku(board): # Initialize sets for rows, columns, and boxes rows = {} cols = {} boxes = {} # Iterate over each cell in the board for r in range(9): for c in range(9): </pre>	<pre> # Simply test def isValidSudoku(board): # Initialize sets for rows, columns, and boxes rows = {} cols = {} boxes = {} # Iterate over each cell in the board for r in range(9): for c in range(9): </pre>	<pre> return True # Example usage: if __name__ == '__main__': board = [['5', '3', '4', '6', '7', '8', '9', '2', '1', '0'], </pre>	<pre> class Solution: def isValidSudoku(self, board: List[List[str]]) -> bool: row = {} col = {} box = {} for i in range(9): for j in range(9): c = board[i][j] if c == '.': continue </pre>						
Matrix	n.1423	Matrix	n.1424	Matrix	n.1425	Matrix	n.1426	Matrix	n.1427	Matrix	n.1428

Matrix

#48 Rotate Image

LeetCode - SimpleLen - WalkCC - LeeCode
Array - Math - Heuristic
Lists: Grid 169 | CodeSignal

Time Complexity: O(n^2) since every element is processed during the transpose and row reversal. Space Complexity: O(1) as the operations are performed in-place without extra data structures.

Problem
You are given an n x n 2D matrix representing an image, rotate the image by 90 degrees clockwise.
You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.
Example 1:

1	2	3	7	4	1
4	5	6	8	5	2
7	8	9	9	6	3

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11	15	13	2	5
2	4	8	10	14	3	4	1
13	3	6	7	12	6	8	9
15	14	12	16	16	7	10	11

Brute Force Approach

```

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        # Brute Force O(n^2) space - copy to new matrix with rotated indices
        n = len(matrix)
        copy = [[]] * n
        for i in range(n):
            for j in range(n):
                copy[i][n-j-1] = matrix[j][i]
    
```

Community Solutions (Python)

WalkCC

```

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        matrix.reverse()
        for i, j in itertools.combinations(range(len(matrix)), 2):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    
```

LeeCode

```

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n // 2):
            for j in range(i):
                matrix[i][j], matrix[n-1-i][j] = matrix[n-1-i][j], matrix[i][j]
            for j in range(n // 2 + i, n - i):
                matrix[i][j], matrix[n-1-i][j] = matrix[n-1-i][j], matrix[i][j]
        matrix.reverse()
    
```

Matrix #54 Spiral Matrix MED LeetCode - SimplyLeet - WalkCC - LeetCode Array - Matrix - Simulation Lists: Grind 169 Codesignal Time Complexity: $O(m \times n)$ since each element is visited exactly once. Space Complexity: $O(1)$ extra space (excluding the space required for the output list). Problem Given an $m \times n$ matrix, return all elements of the matrix in spiral order. Example 1: Input: matrix = [[1,2,3],[4,5,6],[7,8,9]] Output: [1,2,3,6,9,8,7,4,5] Example 2: Matrix p.1435	Input: matrix = [[1,2,3],[4,5,6],[7,8,9]] Output: [1,2,3,6,9,8,7,4,5] Example 2: Matrix p.1436	Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]] Output: [1,2,3,4,7,8,11,12,9,6,5,10] Constraints: $m == \text{matrix.length}$ $n == \text{matrix}[0].\text{length}$ $1 \leq m, n \leq 10$ $-100 \leq \text{matrix}[i][j] \leq 100$ Bruce Force (Python) Matrix Sheet 40/86 - mini-pages 406-1440 - LeetMastery 36-up	# Bruce Force Approach class Solution: <pre>def spiralOrder(self, matrix): # Bruce Force O(m*n) - track visited cells, # simulate turning if not matrix: return [] m, n = len(matrix), len(matrix[0]) directions = [(0,1),(1,0),(0,-1),(-1,0)] # right down left up visited = [[False]*n for _ in range(m)] r,c,d = 0,0,0 result.append(matrix[r][c]) while True: visited[r][c] = True dr,dc = directions[d] nr,nc = r+dr,c+dc if 0 <= nr < m and 0 <= nc < n and not visited[nr][nc]: r,nc = nr,nc d = (d+1)%4 result.append(matrix[r][c]) else: r,c,d = r-1,c,d d = (d+3)%4 result.append(matrix[r][c]) if r == m-1 and c == n-1: return result</pre> Community Solutions (Python) • WalkCC Matrix p.1438	<pre>class Solution: def spiralOrder(self, matrix: List[List[int]]) -> List[int]: if not matrix: return [] m = len(matrix) n = len(matrix[0]) ans = [] r1 = 0 c1 = 0 r2 = m - 1 c2 = n - 1 # Repeatedly add matrix[r1..r2][c1..c2] to 'ans' while len(ans) < m * n: j = c1 while j <= c2 and len(ans) < m * n: ans.append(matrix[r1][j]) j += 1 i = r1 + 1 while i <= r2 and len(ans) < m * n: ans.append(matrix[i][c2]) i += 1 j = c2 while j >= c1 and len(ans) < m * n: ans.append(matrix[r2][j]) j -= 1 i = r2 - 1 while i >= r1 + 1 and len(ans) < m * n: ans.append(matrix[i][c1]) i -= 1 return ans</pre> Matrix p.1439	<pre>while i <= r2 - 1 and len(ans) < m * n: ans.append(matrix[i][c2]) i += 1 j = c2 while j >= c1 and len(ans) < m * n: ans.append(matrix[r2][j]) j -= 1 i = r2 - 1 while i >= r1 + 1 and len(ans) < m * n: ans.append(matrix[i][c1]) i -= 1 return ans</pre> Matrix p.1440
--	---	--	---	--	--

Matrix	p.1441	Matrix	p.1442	Matrix	p.1443	Matrix	p.1444	Matrix	p.1445	Matrix	p.1446
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

Matrix	p.1447	Matrix	p.1448	Matrix	p.1449	Matrix	p.1450	Matrix	p.1451	Matrix	p.1452
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

Matrix	p.1453	Matrix	p.1454	Matrix	p.1455	Matrix	p.1456	Matrix	p.1457	Matrix	p.1458
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

Matrix	p.1459	Matrix	p.1460	Matrix	p.1461	Matrix	p.1462	Matrix	p.1463	Matrix	p.1464
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

Matrix	p.1465	Matrix	p.1466	Matrix	p.1467	Matrix	p.1468	Matrix	p.1469	Matrix	p.1470
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

Matrix	p.1471	Matrix	p.1472	Matrix	Sheet 41/86 · mini-pages 2441-2470 · LeetMastery 36-up	p.1474	Matrix	p.1475	Matrix	p.1476
--------	--------	--------	--------	--------	--	--------	--------	--------	--------	--------

Linked List

#1171 Remove Zero Sum Consecutive Nodes from Linked List

LeetCode - SimplifyLeet - WalkCC - LeetCode

Hash Table - Linked List

Lists Denny Zhang

Time Complexity: O(n) - Two passes over the list. Space Complexity: O(n) - Hash table storing at most n prefix sums.

Problem

Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences.

After doing so, return the head of the final linked list. You may return any such answer.

(Note that in the examples below, all sequences are serializations of ListNode objects.)

Example 1:

Input: head = [1,2,-3,3,1]

Output: [1,2,1]

Note: The answer [1,2,1] would also be accepted.

Example 2:

p1329

Linked List

Input: head = [1,2,3,-3,4]

Output: [1,2,4]

Output: [1]

Example 3:

Input: head = [1,2,3,-3,-2]

Output: [1]

Constraints:

• The given linked list will contain between 1 and 1000 nodes.

• Each node in the linked list has -1000 <= node.val <= 1000.

Community Solutions (Python)

• WalkCC

class Solution:

def removeZeroSumSublists(self, head: ListNode) -> ListNode:

dummy = ListNode(0, head)

prefixToNode = {}

pre = dummy

while head:

pre.next = head

pre = head

pre.next = head

pre = head

p1370

Linked List

prefix = 0

head = dummy

while head:

pre = head

head.next = pre

head = head.next

return dummy.next

p1371

Linked List

LeetCode

• Definition for singly-linked list.

class ListNode:

def __init__(self, val=0, next=None):

self.val = val

self.next = next

class Solution:

def removeZeroSumSublists(self, head: Optional[ListNode]) -> Optional[ListNode]:

dummy = ListNode(next=head)

last = {}

s, cur = 0, dummy

p1373

Linked List

while cur:

s = cur.val

last[s] = cur

s, cur = 0, dummy

while cur:

s = cur.val

cur.next = last[s].next

cur = cur.next

return dummy.next

p1374

Linked List

Linked List

#25 Reverse Nodes in k-Group

LeetCode - SimplifyLeet - WalkCC - LeetCode

Linked List - Recursion

Lists Grind 169

Time Complexity: O(n) where n is the number of nodes (each node is processed a constant number of times)

Space Complexity: O(1) (only constant extra space is used)

Problem

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:

p1375

Linked List

Input: head = [1,2,3,4,5], k = 2

Output: [2,1,4,3,5]

Example 2:

Input: head = [1,2,3,4,5], k = 3

Output: [3,2,1,4,5]

Constraints:

• The number of nodes in the list is n.

1 <= k <= n <= 5000

0 <= Node.val <= 1000

p1376

Linked List

Follow-up: Can you solve the problem in O(1) extra memory space?

Brute Force (Python)

• Brute Force Approach

class Solution:

def reverseKGroup(self, curr, k):

• Brute Force - collect into list, manipulate, rebuild

vals = []

curr = curr.next

while curr:

vals.append(curr.val)

curr = curr.next

dummy = ListNode(0)

curr = dummy

for v in vals:

curr.next = ListNode(v)

curr = curr.next

return dummy.next

p1377

Linked List

Community Solutions (Python)

• WalkCC

p1378

Linked List

class Solution:

def reverseKGroup(self, head: ListNode | None, k: int) -> ListNode | None:

if not head:

return None

if not head:

return None

tail = head

for _ in range(k):

• There are less than k nodes in the list, do nothing.

if not tail:

return head

tail = tail.next

return head

tail = tail.next

newhead = self.reverse(head, tail)

head.next = self.reverseKGroup(tail, k)

return newhead

def reverse(self, head: ListNode | None, tail: ListNode | None):

self, head: ListNode | None,

p1379

Linked List

tail: ListNode | None.

-> ListNode | None:

"""Reverse head, tail."""

prev = None

curr = head

while curr in tail:

next = curr.next

curr.next = prev

prev = curr

curr = next

return prev

class Solution:

def reverseKGroup(self, head: ListNode, k: int) -> ListNode:

if not head or k == 1:

return head

def getLength(head: ListNode) -> int:

length = 0

while head:

length += 1

head = head.next

p1379

Linked List

return length

length = getLength(head)

dummy = ListNode(0, head)

prev = dummy

curr = head

for _ in range(length // k):

for i in range(k - 1):

next = curr.next

curr.next = next.next

prev.next = prev.next

prev = curr

curr = curr.next

return dummy.next

p1379

Linked List

• LeetCode

p1379

Linked List

• Definition for singly-linked list.

class ListNode:

def __init__(self, val=0, next=None):

self.val = val

self.next = next

class Solution:

def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:

Optional[ListNode]:

dummy = ListNode(0)

curr = head

while curr:

next = curr.next

curr.next = dummy.next

dummy.next = curr

curr = next

return dummy.next

dummy = pre = ListNode(next=head)

while pre:

curr = pre

p1374

Linked List

for _ in range(k):

curr = curr.next

if curr is None:

return dummy.next

node = pre.next

curr.next = node

curr = node

pre.next = reverse(node)

node.next = next

pre = node

return dummy.next

p1374

Linked List

• SimplifyLeet

• Definition for singly-linked list.

class ListNode:

def __init__(self, val=0, next=None):

self.val = val

self.next = next

class Solution:

def reverseKGroup(self, head: ListNode, k: int) -> ListNode:

• Create a dummy node to simplify edge cases

dummy = ListNode(0)

p1374

Linked List

dummy.next = head

group_prev = dummy

while True:

• Find the kth node from group_prev

kth = self.getKthNode(group_prev, k)

if not kth:

break

group_next = kth.next

• Reverse the group starting at group_prev.next up to kth

prev = kth.next

curr = group_prev.next

while curr != group_next:

temp = curr.next

curr.next = prev

prev = curr

curr = temp

• Reconnect the reversed group with the previous part

p1374

Linked List

temp = group_prev.next # temp is the start of the original group, becomes the new tail

group_prev.next = kth # Connect previous part to the kth node (new head of reversed group)

group_prev = temp # Move group_prev to the end of the reversed group

return dummy.next

• Helper function to get the kth node from the current node

def getKthNode(self, curr, k):

while curr and k > 0:

curr = curr.next

k -= 1

return curr

p1374

Linked List

• i.e. LeetCode

p1374

Linked List

• Definition for singly-linked list.

class ListNode:

def __init__(self, val=0, next=None):

self.val = val

self.next = next

class Solution:

def reverseKGroup(self, head: ListNode, k: int) -> ListNode:

like

1->None, 2->3->4->5

2->1->None, 3->4->5

3->2->1->None, 4->5

4->3->2->1->None, 5

5->4->3->2->1->None, None

def reverseList(head):

pre, p = None, head

while p:

p.next = pre

pre = p

p = p.next

p1374

Linked List

pre = p

p = p.next

return pre

dummy = ListNode(next=head)

pre = dummy

while pre:

for _ in range(k):

curr = curr.next

curr.next = pre

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

pre = curr

<

class Solution: def cloneGraph(self, node: Optional[Node]) -> Optional[Node]: def dfs(node): if node is None: return None if node in q: return clone(node) clone = Node(node.val) clone.neighbors = clone for nei in node.neighbors: clone.neighbors.append(dfs(nei)) return clone q = defaultdict() return dfs(node) • SimplyLeeT p.1837	# Definition for Node. class Node: def __init__(self, val, neighbors=None): self.val = val # Initialize neighbors with empty list if None is provided self.neighbors = neighbors if neighbors is not None else [] def cloneGraph(node: Node) -> Node: # Hash map to track original nodes and their corresponding clones old_to_new = {} def dfs(current): # Base case: if the current node is None, return None if current is None: return None # If the node has already been cloned, return the clone to avoid duplication and cycle issues if current in old_to_new: return old_to_new[current] # Create a clone for the current node copy = Node(current.val) p.1838	old_to_new[current] = copy # Recursively clone all neighbors for neighbor in current.neighbors: copy.neighbors.append(dfs(neighbor)) return copy return dfs(node) p.1839	def cloneGraph(self, node: Node) -> Node: visited = defaultdict() def dfs(node): if node is None: return None if node in visited: return visited[node] c = Node(node.val) for e in node.neighbors: c.neighbors.append(dfs(e)) return c return dfs(node) p.1840	DFS #200 Number of Islands LeetCode - SimplyLeeT - WalkCC - LeetCode Array - DFS - BFS - Union Find - Matrix Lists: Grid 169 Time Complexity: O(m * n), as each cell is visited at most once. Space Complexity: O(m * n) in the worst-case scenario due to recursion stack (DFS) or auxiliary data structures. Problem Given an m x n 2D binary grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water. Example 1: p.1841	Input: grid = [["1","1","1","1","0"], ["1","1","0","1","0"], ["1","1","0","0","0"], ["0","0","0","0","0"]] Output: 1 Example 2: Input: grid = [["1","1","0","1","0"], ["1","1","0","0","0"], ["0","0","1","0","0"], ["0","0","0","1","1"]] Output: 3 Constraints: m == grid.length n == grid[0].length 1 <= m, n <= 300 grid[i][j] is '0' or '1'. Brute Force (Python) p.1842
# Brute Force Approach class Solution: def numIslands(self, grid): # Brute Force O(m^2*n^2) - DFS from every unvisited land cell if not grid: return 0 n, m = len(grid), len(grid[0]) visited = set() def dfs(r, c): if (r,c) in visited or grid[r][c]=='0': return visited.add((r,c)) for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]: dfs(r+dr, c+dc) count += 1 for c in range(n): if grid[r][c]=='1' and (r,c) not in visited: dfs(r, c) count += 1 return count Community Solutions (Python) • WalkCC p.1843	class Solution: def numIslands(self, grid: List[List[str]]) -> int: DIRS = [(0, 1), (1, 0), (0, -1), (-1, 0)] n, m = len(grid), len(grid[0]) n, m = len(grid), len(grid[0]) visited = set() def dfs(r, c): q = collections.deque([(r, c)]) grid[r][c] = '2' # Mark '2' as visited. while q: i, j = q.popleft() for dx, dy in DIRS: x = i + dx y = j + dy if x < 0 or x == n or y < 0 or y == m: continue if grid[x][y] == '1': continue q.append((x, y)) grid[x][y] = '2' # Mark '2' as visited. p.1844	ans = 0 for i in range(n): for j in range(m): if grid[i][j] == '1': bfs(i, j) ans += 1 return ans class Solution: def numIslands(self, grid: List[List[str]]) -> int: n, m = len(grid) n, m = len(grid[0]) def dfs(i, j): if i < 0 or i == n or j < 0 or j == m: return if grid[i][j] == '1': return p.1845	grid[i][j] == '2' # Mark '2' as visited. dfs(i + 1, j) dfs(i - 1, j) dfs(i, j + 1) dfs(i, j - 1) ans = 0 for i in range(n): for j in range(m): if grid[i][j] == '1': dfs(i, j) ans += 1 return ans p.1846	class Solution: def numIslands(self, grid: List[List[str]]) -> int: def dfs(i, j): grid[i][j] = '0' for a, b in pairwise(dirs): x, y = i + a, j + b if 0 <= x < n and 0 <= y < n and grid[x][y] == '1': dfs(x, y) ans = 0 dirs = (-1, 0), (1, 0, -1) n, m = len(grid), len(grid[0]) for i in range(n): for j in range(m): if grid[i][j] == '1': dfs(i, j) return ans p.1847	# Function to count the number of islands in the grid def numIslands(grid): # If the grid is empty, return 0 if not grid: return 0 rows, cols = len(grid), len(grid[0]) islands = 0 # Depth-First Search helper to mark the entire island as visited def dfs(r, c): # Check if the current cell is out of bounds or water if r < 0 or r > rows or c < 0 or c > cols or grid[r][c] == "0": return # Mark the cell as visited by setting it to water grid[r][c] = "0" # Explore all four adjacent cells (up, down, left, right) dfs(r + 1, c) dfs(r - 1, c) dfs(r, c + 1) dfs(r, c - 1) p.1848
# Iterate over every cell in the grid for i in range(rows): for j in range(cols): # If a land cell is found, conduct DFS to visit the entire island if grid[i][j] == "1": islands += 1 dfs(i, j, j) return islands p.1849	ans = 0 dirs = (-1, 0), (1, 0, -1) n, m = len(grid), len(grid[0]) for i in range(n): for j in range(m): if grid[i][j] == '1': dfs(i, j) ans += 1 return ans p.1850	n, m = len(grid), len(grid[0]) directions = [(0, 1), (0, -1), (1, 0), (-1, 0)] islands = 0 for i in range(n): for j in range(m): if grid[i][j] == "1": islands += 1 q = deque([(i, j)]) # No need to reset q, already drained from previous bfs while q: x, y = q.popleft() for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < n and 0 <= ny < m and grid[nx][ny] == "1": q.append((nx, ny)) grid[nx][ny] = "0" # mark as visited return islands p.1851	# union find # similar to UF in https://leetcode.com/2016-09-30-365-Number-of-Islands-II/ class Solution: def numIslands(self, grid: List[List[str]]) -> int: def check(i, j): return 0 <= i < m and 0 <= j < n and grid[i][j] == "1" # Check for "1" instead of 1 def find(x): if p[x] == x: return p[x] p[x] = find(p[x]) return p[x] n, m = len(grid), len(grid[0]) p = list(range(n * m)) cur = 0 # Initialize cur as 0 instead of n for i in range(n): for j in range(m): if grid[i][j] == "1": cur += 1 # Increment cur when encountering "1" p.1852	for x, y in [(-1, 0), (1, 0), (0, -1), (0, 1)]: if check(i + x, j + y) and find(i + x, j + y) != find(i + x, j + y): find((i + x, j + y)) cur += 1 return cur p.1853	class Solution(object): def numIslands(self, grid): ... :type grid: List[List[str]] :type int ... visited = set() ans = 0 def dfs(grid, i, j, visited): ... add((i, 3), (3, 2)) add((1, 1), (3, 2), (2, 2)) add((1, 1), 3) in a True add((10,18) in a False ... p.1854
if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]) or grid[i][j] == "0" or (i, j) in visited: return False return False visited = {(i, j)} for dr, dj in [(-1, 0), (1, 0), (0, 1), (0, -1)]: newd, newd = i + dr, j + dj dfs(grid, newd, newd, visited) return True for i in range(0, len(grid)): for j in range(0, len(grid[0])): if dfs(grid, i, j, visited): ans += 1 return ans p.1855	DFS #207 Course Schedule LeetCode - SimplyLeeT - WalkCC - LeetCode DFS - BFS - Graph - Topological Sort Lists: Grid 169 Time Complexity: O(V + E) Space Complexity: O(V + E) Problem There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai. Example 1: Input: numCourses = 2, prerequisites = [[1,0]] Output: true Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible. Example 2: Input: numCourses = 2, prerequisites = [[1,0]] Output: false Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is impossible. p.1856	Input: numCourses = 2, prerequisites = [[1,0],[0,1]] Output: false Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible. Constraints: 1 <= numCourses <= 2000 0 <= prerequisites.length <= 5000 prerequisites[i].length == 2 0 <= ai, bi < numCourses All the pairs prerequisites[i] are unique. Brute Force (Python) # Brute Force Approach class Solution: def canFinish(self, numCourses, prerequisites): # Brute Force O(V+E) - DFS cycle detection on adjacency list from collections import defaultdict graph = defaultdict(list) for a, b in prerequisites: graph[b].append(a) # BFS/visited, 1-in-stack, 2-done state = [0] * numCourses p.1857	def has_cycle(node): if state[node] == 1: return True if state[node] == 2: return False state[node] = 1 for nei in graph[node]: if has_cycle(nei): return True state[node] = 2 return False return not any(has_cycle(c) for c in range(numCourses)) p.1858	def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool: graph = {} for i in range(numCourses): graph[i] = [] for v, u in prerequisites: graph[u].append(v) def hasCycle(u: int) -> bool: if state[u] == State.VISITING: return True if state[u] == State.VISITED: return False state[u] = State.VISITING if any(hasCycle(v) for v in graph[u]): return True state[u] = State.VISITED return False return not any(hasCycle(i) for i in range(numCourses)) p.1859	class Solution: def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool: g = {} for i in range(numCourses): g[i] = [] for i in range(numCourses): for j in g[i]: if indegree[j] == 0: if indegree[j] == 0: q.append(j) return numCourses == 0 p.1860
from collections import deque def canFinish(numCourses, prerequisites): # Create a List to hold the number of prerequisites (indegree) for each course. indegree = [0] * numCourses # Create an adjacency list for each course. graph = {} for i in range(numCourses): graph[i] = [] # Build the graph and update indegree for each course. for course, prereq in prerequisites: graph[prereq].append(course) indegree[course] += 1 # Initialize a queue with courses that have no prerequisites. queue = deque([i for i in range(numCourses) if indegree[i] == 0]) # Counter for courses that can be completed. visited = 0 while queue: p.1861	course = queue.popleft() # Remove course with no remaining prerequisites. visited += 1 # Increment count of completed courses. # For each neighbor course, reduce its indegree. for neighbor in graph[course]: indegree[neighbor] -= 1 # Remove dependency. if indegree[neighbor] == 0: queue.append(neighbor) # If we processed all courses, it's possible to finish. return visited == numCourses # Example usage: print(canFinish(2, [(1,0)])) # Expected output: True print(canFinish(2, [(1,0),(0,1)])) # Expected output: False p.1862	''' from collections import defaultdict >>> a = defaultdict(int) >>> a['hehehe'] = 1 >>> a['hehehe'] = 1 >>> a defaultdict(int, {'hehehe': 1}) while q: >>> a = defaultdict(list) >>> a['hehehe'] = ['hehehe'] >>> a defaultdict(list, {'hehehe': ['hehehe']}) p.1863	class Solution: def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]: graph = {} for a, b in prerequisites: graph[b].append(a) indegree = [0] * numCourses graph = {} for i in range(numCourses): graph[i] = [] for v, u in prerequisites: graph[u].append(v) indegree[u] += 1 # Perform topological sorting. q = collections.deque([i for i, d in enumerate(indegree) if d == 0]) while q: p.1864	DFS #210 Course Schedule II LeetCode - SimplyLeeT - WalkCC - LeetCode DFS - BFS - Graph - Topological Sort Lists: Grid 169 Time Complexity: O(V + E) where V is the number of courses and E is the number of prerequisite pairs. Space Complexity: O(V + E) for storing the graph and in-degree array. Problem There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai. For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1. Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array. Example 1: p.1865	Input: numCourses = 2, prerequisites = [[1,0]] Output: [0,1] Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1]. Example 2: Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]] Output: [0,2,3,1] Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0. So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3]. Example 3: Input: numCourses = 1, prerequisites = [] Output: [0] Constraints: 1 <= numCourses <= 2000 0 <= prerequisites.length <= numCourses * (numCourses - 1) prerequisites[i].length == 2 0 <= ai, bi < numCourses ai != bi All the pairs [ai, bi] are distinct. Brute Force (Python) p.1866
# Brute Force Approach class Solution: def findOrder(self, numCourses, prerequisites): # Brute Force O(V+E) - DFS topological sort from collections import defaultdict graph = defaultdict(list) for a, b in prerequisites: graph[b].append(a) state = [0] * numCourses result = [] def dfs(node): if state[node] == 1: return False # cycle if state[node] == 2: return True for nei in graph[node]: if not dfs(nei): return False state[node] = 2 result.append(node) return True for c in range(numCourses): if not dfs(c): return [] return result[::-1] Community Solutions (Python) • WalkCC p.1867	from enum import Enum class State(Enum): INIT = 0 VISITED = 1 VISITED = 2 class Solution: def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]: ans = [] graph = {} for i in range(numCourses): graph[i] = [] for v, u in prerequisites: graph[u].append(v) indegree = [0] * numCourses graph = {} for i in range(numCourses): graph[i] = [] for v, u in prerequisites: graph[u].append(v) indegree[u] += 1 # Perform topological sorting. q = collections.deque([i for i, d in enumerate(indegree) if d == 0]) while q: p.1868	graph[u].append(v) def hasCycle(u: int) -> bool: if state[u] == State.VISITING: return True if state[u] == State.VISITED: return False state[u] = State.VISITING if any(hasCycle(v) for v in graph[u]): return True state[u] = State.VISITED ans.append(u) return False if any(hasCycle(i) for i in range(numCourses)): return [] return ans[::-1] p.1869	class Solution: def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]: graph = {} for a, b in prerequisites: graph[b].append(a) indegree = [0] * numCourses graph = {} for i in range(numCourses): graph[i] = [] for v, u in prerequisites: graph[u].append(v) indegree[u] += 1 # Perform topological sorting. q = collections.deque([i for i, d in enumerate(indegree) if d == 0]) while q: p.1870	u = q.popleft() ans.append(u) for v in graph[u]: indegree[v] -= 1 if indegree[v] == 0: q.append(v) return ans if len(ans) == numCourses else [] p.1871	i = q.popleft() ans.append(i) for j in g[i]: indegree[j] -= 1 if indegree[j] == 0: q.append(j) return ans if len(ans) == numCourses else [] p.1872
# Build the graph. for v, u in prerequisites: graph[u].append(v) indegree[v] += 1 # Perform topological sorting. q = collections.deque([i for i, d in enumerate(indegree) if d == 0]) while q: p.1873	class Solution: def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]: graph = {} for a, b in prerequisites: graph[b].append(a) indegree = [0] * numCourses graph = {} for i in range(numCourses): graph[i] = [] for v, u in prerequisites: graph[u].append(v) indegree[u] += 1 # Perform topological sorting. q = collections.deque([i for i, d in enumerate(indegree) if d == 0]) while q: p.1874	class Solution: def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]: graph = {} for a, b in prerequisites: graph[b].append(a) indegree = [0] * numCourses graph = {} for i in range(numCourses): graph[i] = [] for v, u in prerequisites: graph[u].append(v) indegree[u] += 1 # Perform topological sorting. q = collections.deque([i for i, d in enumerate(indegree) if d == 0]) while q: p.1875	# Python solution using BFS (Kahn's Algorithm) from collections import deque def findOrder(numCourses, prerequisites): # Build the graph and the in-degree array in_degree = [0] * numCourses graph = {} for a, b in prerequisites: graph[prereq].append(course) in_degree[course] += 1 p.1876		

#685 Redundant Connection II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - BFS - Union Find - Graph Lists: Denny Zhang Time Complexity: O(n) Space Complexity: O(n) Problem In this problem, a rooted tree is a directed graph such that there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents. The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n, and was not an edge that already existed. The resulting graph is given as a 2D-array of edges. Each element of edges is a pair [u, v] that represents a directed edge connecting nodes u and v, where u is a parent of child v. Return an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs last in the given 2D-array. Example 1: Input: edges = [[1,2],[1,3],[2,3]] Output: [2,3] Example 2: Input: edges = [[1,2],[2,3],[3,4],[4,1],[1,5]] Output: [4,1] Constraints: - n == edges.length 3 <= n <= 1000 - edges[i].length == 2 1 <= u, v <= n - u != v Brute Force (Python) <pre>class Solution: def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]: # Brute Force - DFS from every unvisited node visited = set() count = 0 def dfs(node): visited.add(node) for nb in graph.get(node, []): if nb not in visited: dfs(nb) for node in range(n): if node not in visited: dfs(node); count += 1 return count</pre> p2017		p2018	#192 Critical Connections in a Network LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Backtracked Component Lists: Denny Zhang Time Complexity: O(n * m) where n is the number of nodes and m is the number of connections. Space Complexity: O(n * m) for the graph representation and DFS recursion stack. Problem A network of n servers numbered from 0 to n - 1 connected by undirected server-to-server connections forming a network where connections[i] = [ai, bi] represents a connection between servers ai and bi. Any server can reach other servers directly or indirectly through the network. A critical connection is a connection that, if removed, will make some servers unable to reach some other server. Return all critical connections in the network in any order. Example 1: Input: n = 4, connections = [[0,1],[1,2],[2,0],[1,3]] Output: [[1,3]] Explanation: [1,3] is also accepted. Example 2: Input: n = 2, connections = [[0,1]] Output: [[0,1]] Constraints: 2 <= n <= 105 - n - 1 <= connections.length <= 105 0 <= ai, bi <= n - 1 Community Solutions (Python) - WalkCC p2019		p2020	#207 Course Schedule LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2021	<pre>class UnionFind: def __init__(self, n: int): self.id = list(range(n)) self.rank = [0] * n def union(self, u: int, v: int) -> bool: u = self.find(u) v = self.find(v) if u == v: return False if self.rank[u] < self.rank[v]: self.id[u] = v elif self.rank[u] > self.rank[v]: self.id[v] = u else: self.id[u] = v self.rank[u] = 1 return True def find(self, u: int) -> int: if self.id[u] == u: return self.id[u] self.id[u] = self.find(self.id[u]) return self.id[u]</pre>	p2022
#1036 Escape a Large Maze LeetCode - SimplifyLeet - WalkCC - LeetCode Array - Hash Table - DFS - BFS Lists: Denny Zhang Time Complexity: O(n^2) in the worst case, where n is the number of blocked cells (with n <= 2000). Space Complexity: O(n^2) due to the storage of visited cells in the worst-case scenario. Problem There is a million by 1 million grid on an XY-plane, and the coordinates of each grid square are (x, y). We start at the source = [sx, sy] square and want to reach the target = [tx, ty] square. There is also an array of blocked squares, where each blocked[i] = [xi, yi] represents a blocked square with coordinates (xi, yi). Each move, we can walk one square north, east, south, or west if the square is not in the array of blocked squares. We are also not allowed to walk outside of the grid. Return true if and only if it is possible to reach the target square from the source square through a sequence of valid moves. Example 1: Input: blocked = [[0,1],[1,0]], source = [0,0], target = [0,2] Output: false Explanation: The target square is inaccessible starting from the source square because we cannot move. We cannot move north or east because those squares are blocked. We cannot move south or west because we cannot go outside of the grid. Example 2: Input: blocked = [], source = [0,0], target = [0,1000000000] Output: true Explanation: Because there are no blocked cells, it is possible to reach the target square. Constraints: 0 <= blockedLength <= 200 - blocked[i].length == 2 0 <= xi, yi <= 10^6 - source.length == target.length == 2 0 <= sx, sy, tx, ty <= 10^6 - source != target - It is guaranteed that source and target are not blocked. Brute Force (Python) <pre>class Solution: def isEscapePossible(self, blocked: List[List[int]], source: List[int], target: List[int]) -> bool: # Brute Force - DFS from every unvisited node visited = set() count = 0 def dfs(node): visited.add(node) for nb in graph.get(node, []): if nb not in visited: dfs(nb) for node in range(n): if node not in visited: dfs(node); count += 1 return count</pre> p2023	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2024	#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2025	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2026			
#733 Flood Fill LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Flood Fill Lists: Denny Zhang Time Complexity: O(n * m) where n and m are the dimensions of the image. Problem You are given an image represented by an n x m grid of pixels. Each pixel has a color value between 0 and k - 1. You are given a starting pixel (sx, sy) and a new color. Return the image after the flood fill operation. Example 1: Input: image = [[1,1,1],[1,1,0],[1,0,1]], sx = 0, sy = 0, newColor = 2 Output: [[2,2,2],[2,2,0],[2,0,1]] Example 2: Input: image = [[0,0,0],[0,0,0]], sx = 0, sy = 0, newColor = 0 Output: [[0,0,0],[0,0,0]] Constraints: - n == image.length - m == image[0].length 0 <= sx < n 0 <= sy < m 0 <= newColor <= k - 1 Community Solutions (Python) - LeetCode p2027	<pre>def dfs(x, y): if (x, y) in visited: return visited.add((x, y)) for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < n and 0 <= ny < m and image[nx][ny] == image[x][y]: dfs(nx, ny)</pre>	p2028						
#200 Number of Islands LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Flood Fill Lists: Denny Zhang Time Complexity: O(n * m) where n and m are the dimensions of the grid. Problem You are given an m x n binary grid representing an island map. Return the number of islands. Example 1: Input: grid = [[1,1,1,1,0],[1,1,0,1,0],[1,1,0,0,0],[0,0,0,0,0]] Output: 1 Example 2: Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,0,0]] Output: 1 Example 3: Input: grid = [[1,0,1,0,1],[1,0,1,0,1],[0,0,0,0,0]] Output: 3 Constraints: - m == grid.length - n == grid[0].length 1 <= m, n <= 300 - grid[i][j] is either 0 or 1 Community Solutions (Python) - LeetCode p2029	<pre>def dfs(x, y): if (x, y) in visited: return visited.add((x, y)) for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == 1: dfs(nx, ny)</pre>	p2030						
#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2031	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2032						
#200 Number of Islands LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Flood Fill Lists: Denny Zhang Time Complexity: O(n * m) where n and m are the dimensions of the grid. Problem You are given an m x n binary grid representing an island map. Return the number of islands. Example 1: Input: grid = [[1,1,1,1,0],[1,1,0,1,0],[1,1,0,0,0],[0,0,0,0,0]] Output: 1 Example 2: Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,0,0]] Output: 1 Example 3: Input: grid = [[1,0,1,0,1],[1,0,1,0,1],[0,0,0,0,0]] Output: 3 Constraints: - m == grid.length - n == grid[0].length 1 <= m, n <= 300 - grid[i][j] is either 0 or 1 Community Solutions (Python) - LeetCode p2033	<pre>def dfs(x, y): if (x, y) in visited: return visited.add((x, y)) for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == 1: dfs(nx, ny)</pre>	p2034						
#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2035	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2036						
#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2037	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2038						
#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2039	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2040						
#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2041	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2042						
#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2043	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2044						
#210 Course Schedule II LeetCode - SimplifyLeet - WalkCC - LeetCode DFS - Graph - Topological Sort Lists: Denny Zhang Time Complexity: O(V * E) where V is the number of courses and E is the number of prerequisite pairs. Problem There are several courses numbered from 0 to n - 1. You are given a list of prerequisite pairs (ai, bi) where ai is a prerequisite for bi. Return the order of courses you should take to finish all courses. Example 1: Input: prerequisites = [[2,0],[1,0],[3,1]] Output: [0,1,2,3] Explanation: There are four courses numbered 0 to 3. The prerequisites are (2,0), (1,0), and (3,1). One possible order to finish all courses is 0, 1, 2, 3. Example 2: Input: prerequisites = [[1,0],[0,1],[2,1]] Output: [] Explanation: There are three courses numbered 0 to 2. The prerequisites are (1,0), (0,1), and (2,1). It is impossible to finish all courses because of the cycle (0 -> 1 -> 0). Constraints: 0 <= n <= 1000 0 <= prerequisites.length <= 1000 - ai != bi - There are no repeated connections. Community Solutions (Python) - LeetCode p2045	<pre>def dfs(x, y): if (x, y) in blocked or (x, y) in seen: return False if (x, y) == target: return True for dx, dy in directions: nx, ny = x + dx, y + dy if 0 <= nx < 10^6 and 0 <= ny < 10^6 and (nx, ny) not in blocked and (nx, ny) not in seen: queue.append((nx, ny)) seen.add((nx, ny)) steps += 1 if steps == max_steps: return True return bfs(source, target) and bfs(target, source)</pre>	p2046						

<pre> class BinaryIndexedTree: def __init__(self, n): self.n = n self.c = [0] * (n + 1) @staticmethod def countSmaller(self, nums: List[int]) -> List[int]: alts = sorted(set(nums)) n = (v: i for i, v in enumerate(alts, 1)) tree = BinaryIndexedTree(len(n)) ans = [] for i in nums[::-1]: ans += [i] while x <= self.n: self.c[x] += delta x = BinaryIndexedTree.free.Lowbit(x) def query(self, x): s = 0 while x > 0: s += self.c[x] x = BinaryIndexedTree.free.Lowbit(x) return s # Recursive function for merge sort with counting </pre>	<pre> class Solution: def countSmaller(self, nums: List[int]) -> List[int]: alts = sorted(set(nums)) n = (v: i for i, v in enumerate(alts, 1)) tree = BinaryIndexedTree(len(n)) ans = [] for i in nums[::-1]: ans.append(tree.query(x - 1)) return ans[::-1] # Python solution using merge sort approach def countSmaller(nums): n = len(nums) # Initialize counts array for results counts = [0] * n # Pair each element with its original index enum = [(num, i) for i, num in enumerate(nums)] # Recursive function for merge sort with counting </pre>	<pre> def mergeSort(arr): if len(arr) == 1: return arr # Split the array into two halves mid = len(arr) // 2 left = mergeSort(arr[:mid]) right = mergeSort(arr[mid:]) merged = [] l = r = 0 # r_count counts number of elements taken from right array r_count = 0 while l < len(left) and r < len(right): # If left element is less or equal, it means current right elements are smaller if left[l][0] <= right[r][0]: # update count for original index counts[left[l][1]] += r_count merged.append(left[l]) l += 1 else: # right element is smaller, so increment r_count and add it to merged merged.append(right[r]) r_count += 1 r += 1 while l < len(left): merged.append(left[l]) l += 1 while r < len(right): merged.append(right[r]) r += 1 return merged # Start the merge sort process mergeSort(enum) return counts # Example test print(countSmaller([5, 2, 6, 1])) # Expected output: [2, 3, 3, 0] </pre>	<pre> bisect: maintaining a list in sorted order without having to sort the list after each insertion. https://docs.python.org/3/library/bisect.html bisect.bisect_left() Locate the insertion point for x in a to maintain sorted order. bisect.bisect_right() or bisect.bisect() Similar to bisect_left(), but returns an insertion point which comes after (to the right of) any existing entries of x in a. bisect.insort_left(x, x, lo=0, hi=len(a), *, key=None) Insert x in a in sorted order. Keep in mind that the O(log n) search is dominated by the slow O(n) insertion step. bisect.insort_right(x, x, lo=0, hi=len(a), *, key=None) Similar to bisect_left(), but inserting x in a after any existing entries of x. </pre>	<pre> >>> import bisect >>> bisect.bisect_left([1,2,3], 2) 1 >>> bisect.bisect_right([1,2,3], 2) 2 >>> a = [1, 1, 1, 2, 3] >>> bisect.insort_left(a, 1.0) >>> a [1.0, 1, 1, 1, 2, 3] >>> a = [1, 1, 1, 2, 3] >>> bisect.insort_right(a, 1.0) >>> a [1, 1, 1, 1.0, 2, 3] >>> a = [1, 1, 1, 2, 3] >>> bisect.insort(a, 1.0) >>> a </pre>	
<pre> [1, 1, 1, 1.0, 2, 3] ... import bisect class Solution(object): def countSmaller(self, nums): # nums: List[int] # type: List[int] ans = [] bst = [] for num in reversed(nums): idx = bisect.bisect_left(bst, num) ans.append(idx) bisect.insort(bst, num) return ans[::-1] </pre>	<pre> class BinaryIndexedTree: def __init__(self, n): self.n = n self.c = [0] * (n + 1) @staticmethod def countSmaller(self, nums: List[int]) -> List[int]: alts = sorted(set(nums)) n = (v: i for i, v in enumerate(alts, 1)) tree = BinaryIndexedTree(len(n)) ans = [] for i in nums[::-1]: ans.append(tree.query(x - 1)) return ans[::-1] # Python solution using merge sort approach def countSmaller(nums): n = len(nums) # Initialize counts array for results counts = [0] * n # Pair each element with its original index enum = [(num, i) for i, num in enumerate(nums)] # Recursive function for merge sort with counting </pre>	<pre> return s class Solution: def countSmaller(self, nums: List[int]) -> List[int]: alts = sorted(set(nums)) n = (v: i for i, v in enumerate(alts, 1)) tree = BinaryIndexedTree(len(n)) ans = [] for i in nums[::-1]: ans.append(tree.query(x - 1)) return ans[::-1] # Python solution using merge sort approach def countSmaller(nums): n = len(nums) # Initialize counts array for results counts = [0] * n # Pair each element with its original index enum = [(num, i) for i, num in enumerate(nums)] # Recursive function for merge sort with counting </pre>	<pre> Binary Search #410 Split Array Largest Sum LeetCode - SimplifyLeet - WalkCC - LeetCode Array - Binary Search - Dynamic Programming - Greedy - Prefix Sum Lists Denny Zhang Time Complexity: O(n * log(sum(nums))) - n elements are processed for each binary search iteration. Space Complexity: O(1) - Constant extra space is used. Problem Given an integer array nums and an integer k, split nums into k non-empty subarrays such that the largest sum of any subarray is minimized. Return the minimized largest sum of the split. A subarray is a contiguous part of the array. Example 1: Input: nums = [7,2,5,10,8], k = 2 Output: 18 Explanation: There are four ways to split nums into two subarrays. The best way is to split it into [7,2,5] and [10,8], where the largest sum among the two subarrays is only 18. Example 2: </pre>	<pre> Input: nums = [1,2,3,4,5], k = 2 Output: 9 Explanation: There are four ways to split nums into two subarrays. The best way is to split it into [1,2,3] and [4,5], where the largest sum among the two subarrays is only 9. Constraints: 1 <= nums.length <= 1000 0 <= nums[i] <= 106 1 <= k <= min(50, nums.length) Brute Force (Python) # Brute Force Approach class Solution: def splitArray(self, nums, k): # Brute Force O(n) - Linear scan instead of binary search for i, val in enumerate(nums): if val == target: return i return -1 </pre>	<pre> class Solution: def splitArray(self, nums: List[int], k: int) -> int: prefix = list(itertools.accumulate(nums, initial=0)) @functools.lru_cache(None) def dp(i: int, k: int) -> int: # Returns the minimum of the maximum sum to split the first i numbers into k groups. if k == 1: return prefix[i] return min(max(dp(j, k - 1), prefix[i] - prefix[j]) for j in range(k - 1, i)) return dp(len(nums), k) </pre>
<pre> class Solution: def splitArray(self, nums: List[int], k: int) -> int: n = len(nums) # dp[i][k] is the minimum of the maximum sum to split the first i numbers # into k groups dp = [[math.inf] * (k + 1) for _ in range(n + 1)] prefix = list(itertools.accumulate(nums, initial=0)) for i in range(1, n + 1): dp[i][1] = prefix[i] for l in range(2, k + 1): for i in range(1, n + 1): for j in range(1, i - 1): dp[i][l] = min(dp[i][l], max(dp[j][l - 1], prefix[i] - prefix[j])) return dp[n][k] </pre>	<pre> class Solution: def splitArray(self, nums: List[int], k: int) -> int: n = len(nums) r = sum(nums) + 1 def numGroups(maxSumInGroup) -> int: groupCount = 1 sumInGroup = 0 for num in nums: if sumInGroup + num >= maxSumInGroup: groupCount += 1 sumInGroup = num return groupCount while l < r: m = (l + r) // 2 if numGroups(m) > k: l = m + 1 else: r = m return l </pre>	<pre> class Solution: def splitArray(self, nums: List[int], k: int) -> int: def check(x): s, cnt = inf, 0 for i in range(1, n + 1): s += x if s >= x: cnt += 1 s = 0 return cnt <= k left, right = max(nums), sum(nums) while left < right: mid = (left + right) // 2 if check(mid): right = mid else: left = mid + 1 return left </pre>	<pre> class Solution: def splitArray(self, nums: List[int], k: int) -> int: def check(x): s, cnt = inf, 0 for i in range(1, n + 1): s += x if s >= x: cnt += 1 s = 0 return cnt <= k left, right = max(nums), sum(nums) while left < right: mid = (left + right) // 2 if check(mid): right = mid else: left = mid + 1 return left </pre>		
<pre> class Solution: def findKthNumber(self, n: int, k: int) -> int: left, right = 1, n + 1 while left < right: cnt = 0 for i in range(1, n + 1): cnt += min(n // i, n) if cnt < k: left = mid + 1 else: right = mid return left </pre>	<pre> class Solution: def findKthNumber(self, n: int, k: int) -> int: left, right = 1, n + 1 while left < right: cnt = 0 for i in range(1, n + 1): cnt += min(n // i, n) if cnt < k: left = mid + 1 else: right = mid return left </pre>	<pre> class Solution: def findKthNumber(self, n: int, k: int) -> int: left, right = 1, n + 1 while left < right: cnt = 0 for i in range(1, n + 1): cnt += min(n // i, n) if cnt < k: left = mid + 1 else: right = mid return left </pre>	<pre> class Solution: def findKthNumber(self, n: int, k: int) -> int: left, right = 1, n + 1 while left < right: cnt = 0 for i in range(1, n + 1): cnt += min(n // i, n) if cnt < k: left = mid + 1 else: right = mid return left </pre>	<pre> class Solution: def findKthNumber(self, n: int, k: int) -> int: left, right = 1, n + 1 while left < right: cnt = 0 for i in range(1, n + 1): cnt += min(n // i, n) if cnt < k: left = mid + 1 else: right = mid return left </pre>	<pre> class Solution: def findKthNumber(self, n: int, k: int) -> int: left, right = 1, n + 1 while left < right: cnt = 0 for i in range(1, n + 1): cnt += min(n // i, n) if cnt < k: left = mid + 1 else: right = mid return left </pre>
<pre> Input: ["Solution", "pick", "pick", "pick", "pick", "pick"], [{"r": [2, 3, 5]}, [{"i": [1, 1, 1, 1, 1, 1, 1, 1]}] Output: [null, 0, 4, 1, 6, 1, 0, 4] Explanation Solution solution = new Solution([7, 2, 3, 5]); Constraints: 1 <= n <= 109 0 <= blacklist.length <= min(105, n - 1) 0 <= blacklist[i] < n All the values of blacklist are unique. At most 2 * 10^4 calls will be made to pick. Brute Force (Python) # Brute Force Approach class Solution: def __init__(self, n: int, blacklist: List): # Brute Force O(n) - Linear scan instead of binary search for i, val in enumerate(blacklist): if val == target: return i return -1 </pre>	<pre> class Solution: def __init__(self, n: int, blacklist: List[int]): self.validRange = n - len(blacklist) self.dict = {} maxAvailable = n - 1 for b in blacklist: self.dict[b] = -1 for b in blacklist: if b < self.validRange: # Find the slot that haven't been used. maxAvailable = 1 self.dict[b] = maxAvailable maxAvailable += 1 def pick(self) -> int: value = random.randint(0, self.validRange - 1) if value in self.dict: return self.dict[value] return value </pre>	<pre> class Solution: def __init__(self, n: int, blacklist: List[int]): self.k = n - len(blacklist) self.d = {} i = self.k black = set(blacklist) for b in blacklist: if b < self.k: while i in black: i += 1 def pick(self) -> int: self.d[b] = i i += 1 def pick(self) -> int: value = self.d return self.d.get(x, x) </pre>	<pre> self.d[b] = i i += 1 def pick(self) -> int: x = randrange(self.k) return self.d.get(x, x) # Your Solution object will be instantiated and called as such: obj = Solution(n, blacklist) # param_1 = obj.pick() </pre>		
<pre> Input: ["Solution", "pick", "pick", "pick", "pick", "pick"], [{"r": [2, 3, 5]}, [{"i": [1, 1, 1, 1, 1, 1, 1, 1]}] Output: [null, 0, 4, 1, 6, 1, 0, 4] Explanation Solution solution = new Solution([7, 2, 3, 5]); Constraints: 1 <= n <= 109 0 <= blacklist.length <= min(105, n - 1) 0 <= blacklist[i] < n All the values of blacklist are unique. At most 2 * 10^4 calls will be made to pick. Brute Force (Python) # Brute Force Approach class Solution: def __init__(self, n: int, blacklist: List): # Brute Force O(n) - Linear scan instead of binary search for i, val in enumerate(blacklist): if val == target: return i return -1 </pre>	<pre> class Solution: def __init__(self, n: int, blacklist: List[int]): self.k = n - len(blacklist) self.d = {} i = self.k black = set(blacklist) for b in blacklist: if b < self.k: while i in black: i += 1 def pick(self) -> int: value = random.randint(0, self.validRange - 1) if value in self.dict: return self.dict[value] return value </pre>	<pre> class Solution: def __init__(self, n: int, blacklist: List[int]): self.k = n - len(blacklist) self.d = {} i = self.k black = set(blacklist) for b in blacklist: if b < self.k: while i in black: i += 1 def pick(self) -> int: value = self.d return self.d.get(x, x) </pre>	<pre> self.d[b] = i i += 1 def pick(self) -> int: x = randrange(self.k) return self.d.get(x, x) # Your Solution object will be instantiated and called as such: obj = Solution(n, blacklist) # param_1 = obj.pick() </pre>		
<pre> Input: stations = [23,24,36,46,56,57,65,84,80], k = 1 Output: 14.00000 Constraints: 10 <= stations.length <= 2000 0 <= stations[i] <= 108 stations is sorted in a strictly increasing order. 1 <= k <= 106 # Brute Force Approach class Solution: def minmaxGasDist(self, stations: List): # Brute Force O(n) - Linear scan instead of binary search for i, val in enumerate(stations): if val == target: return i return -1 </pre>	<pre> class Solution: def minmaxGasDist(self, stations: List[int], k: int) -> float: ER = 1e-6 l = 0 r = stations[-1] - stations[0] def possible(k: int, m: float) -> bool: return True if can use m gas stations to ensure that each adjacent distance between gas stations <= m. for a, b in zip(stations, stations[1:]): if diff = b - a if diff == m: k = math.ceil(diff / m) - 1 if k <= 0: return True while r - l > ER: </pre>	<pre> m = (l + r) / 2 if possible(k, m): r = m else: l = m return l # LeetCode class Solution: def minmaxGasDist(self, stations: List[int], k: int) -> float: def check(x): return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k left, right = 0, 1e8 while right - left > 1e-6: mid = (left + right) / 2 if check(mid): right = mid else: left = mid </pre>	<pre> Input: math def minmaxGasDist(stations: List): # Initialize the search range between 0 and the maximum gap between existing stations low = 0.0 high = max(stations[i] - stations[i] for i in range(len(stations)-1)) # Use the binary search with a tolerance of 1e-6 while high - low > 1e-6 mid = (low + high) / 2.0 used = 0 # Calculate additional stations needed for each gap for i in range(1, len(stations)): gap = stations[i] - stations[i-1] used = math.ceil(gap / mid) - 1 # If the used stations are less than or equal to k, try a smaller distance if used <= k: high = mid else: low = mid </pre>	<pre> return high # Example usage: if __name__ == '__main__': print('({:5f}).format(minmaxGasDist([1,2,3,4,5,4,7,8,9,10], 9)) print('({:5f}).format(minmaxGasDist([23,24,36,46,56,57,65,84,80], 1))) # LeetCode class Solution: def minmaxGasDist(self, stations: List[int], k: int) -> float: def check(x): return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k left, right = 0, 1e8 while right - left > 1e-6: mid = (left + right) / 2 if check(mid): right = mid else: left = mid </pre>	
<pre> Binary Search #774 Minimize Max Distance to Gas Station LeetCode - SimplifyLeet - WalkCC - LeetCode Array - Binary Search Lists Denny Zhang Time Complexity: O(n * log(max_gap/precision)), where n is the number of stations. Space Complexity: O(1) (Ignoring input storage) Problem You are given an integer array stations that represents the positions of the gas stations on the x-axis. You are also given an integer k. You should add k new gas stations. You can add the stations anywhere on the x-axis, and not necessarily on an integer position. Let penalty() be the maximum distance between adjacent gas stations after adding the k new stations. Return the smallest possible value of penalty(). Answers within 10^-6 of the actual answer will be accepted. Example 1: Input: stations = [1,2,3,4,5,6,7,8,9,10], k = 9 Output: 0.50000 Example 2: </pre>	<pre> Input: stations = [23,24,36,46,56,57,65,84,80], k = 1 Output: 14.00000 Constraints: 10 <= stations.length <= 2000 0 <= stations[i] <= 108 stations is sorted in a strictly increasing order. 1 <= k <= 106 # Brute Force Approach class Solution: def minmaxGasDist(self, stations: List): # Brute Force O(n) - Linear scan instead of binary search for i, val in enumerate(stations): if val == target: return i return -1 </pre>	<pre> class Solution: def minmaxGasDist(self, stations: List[int], k: int) -> float: ER = 1e-6 l = 0 r = stations[-1] - stations[0] def possible(k: int, m: float) -> bool: return True if can use m gas stations to ensure that each adjacent distance between gas stations <= m. for a, b in zip(stations, stations[1:]): if diff = b - a if diff == m: k = math.ceil(diff / m) - 1 if k <= 0: return True while r - l > ER: </pre>	<pre> m = (l + r) / 2 if possible(k, m): r = m else: l = m return l # LeetCode class Solution: def minmaxGasDist(self, stations: List[int], k: int) -> float: def check(x): return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k left, right = 0, 1e8 while right - left > 1e-6: mid = (left + right) / 2 if check(mid): right = mid else: left = mid </pre>	<pre> Input: math def minmaxGasDist(stations: List): # Initialize the search range between 0 and the maximum gap between existing stations low = 0.0 high = max(stations[i] - stations[i] for i in range(len(stations)-1)) # Use the binary search with a tolerance of 1e-6 while high - low > 1e-6 mid = (low + high) / 2.0 used = 0 # Calculate additional stations needed for each gap for i in range(1, len(stations)): gap = stations[i] - stations[i-1] used = math.ceil(gap / mid) - 1 # If the used stations are less than or equal to k, try a smaller distance if used <= k: high = mid else: low = mid </pre>	<pre> return high # Example usage: if __name__ == '__main__': print('({:5f}).format(minmaxGasDist([1,2,3,4,5,4,7,8,9,10], 9)) print('({:5f}).format(minmaxGasDist([23,24,36,46,56,57,65,84,80], 1))) # LeetCode class Solution: def minmaxGasDist(self, stations: List[int], k: int) -> float: def check(x): return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k left, right = 0, 1e8 while right - left > 1e-6: mid = (left + right) / 2 if check(mid): right = mid else: left = mid </pre>

Binary Search #1235 Maximum Profit in Job Scheduling LeetCode - SimplifyLeet - WalICC - LeetCode Array - Binary Search - Dynamic Programming - Sorting Lists Grid 169 Time Complexity: O(n log n) due to sorting and binary search for each job. Space Complexity: O(n) for the dp arrays. Problem We have n jobs, where every job is scheduled to be done from start_i(time) to end_i(time), obtaining a profit of profit_i. You're given the start_iTime, end_iTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subarray with overlapping time range. If you choose a job that ends at time x you will be able to start another job that starts at time x. Example 1: profit = 40 profit = 10 profit = 50 profit = 70 1 2 3 4 5 6 Input: start₁Time = [1,2,3,3], end₁Time = [3,4,5,6], profit₁ = [50,40,70] Output: 100 Explanation: The subset chosen is the first and fourth job. Time range [1-3]+[3-4] = , we get profits of 100 = 50 + 70. Example 2: profit = 20 profit = 100 profit = 40 1 2 3 4 5 6 7 8 9 10 Input: start₁Time = [1,2,3,4,5], end₁Time = [3,3,18,4,9], profit₁ = [20,20,100,70,40] Output: 110 Explanation: The subset chosen is the first, fourth and fifth job. Profit obtained 110 = 20 + 70 + 40. Binary Search p.2233	Binary Search p.2234	Binary Search p.2235	Binary Search p.2236	Binary Search p.2237	Binary Search p.2238
Binary Search # Example usage: print(jobschedule([1,2,3,3], [3,4,5,6], [50,40,70])) class Solution: def jobschedule(self, start₁Time: List[int], end₁Time: List[int], profit₁: List[int]) -> int: def dfs(i): if i >= n: return 0 if i == jobs[i].start: if i == jobs[i].end: return max(dfs(i + 1), p + dfs(i)) jobs = sorted(zip(start₁Time, end₁Time, profit₁)) n = len(profit) return dfs(0) Binary Search p.2239	Binary Search p.2240	Binary Search p.2241	Binary Search p.2242	Binary Search p.2243	Binary Search p.2244
Binary Search #33 Search in Rotated Sorted Array [Medium] LeetCode - SimplifyLeet - WalICC - LeetCode Array - Binary Search - Dynamic Programming - Sorting Lists Grid 169 Time Complexity: O(log n) per iteration. Space Complexity: O(1). Solution We use an iterative binary search algorithm. We initialize two pointers, left and right, to represent the current segment of the array that could contain the target. At each iteration, we compute the middle index and compare the element there with the target. If they match, we return the index. Otherwise, based on whether the target is smaller or larger than the middle element, we adjust our pointers to narrow the search to the left or right half of the array. This process continues until the target is found or until the pointers cross, indicating that the target is not present. No extra data structures are used, making the space complexity O(1). #33 Search in Rotated Sorted Array [Medium] elements to return the result. This ensures that only relevant hits within the past 5 minutes are counted. The design scales well when hit frequency is high because hits at the same timestamp are grouped together instead of inserting individual records. #528 Random Pick with Weight [Medium] Key Insights • Build a prefix sum array from the weights. • Calculate the cumulative sum of weights up to the current index. • Generate a random number in the range [1, total sum] and use binary search to find the corresponding index in the prefix sum array. • Using binary search ensures efficient selection in O(log n) time for each call of pick(). Space & Time Time Complexity: O(n) for constructing the prefix sum array, and O(log n) per pick(). Space Complexity: O(n) for storing the prefix sum array. Solution The solution leverages a prefix sum array where each entry at index i represents the cumulative sum of weights up to i. After computing the total weight, a random integer is generated between 1 and the total sum inclusive. A binary search is then used to find the first index where the prefix sum is greater than or equal to the random value. #153 Find the Index of the Largest Integer [Medium] Key Insights • Use binary search to efficiently narrow down the position of the unique large value. • At each step, divide the candidate range into two halves (or almost two halves when the range size is odd). Binary Search p.2251	Binary Search p.2252	Binary Search p.2253	Binary Search p.2254	Binary Search p.2249	Binary Search p.2250
Binary Search Space & Time Time Complexity: O(n log n) on average, where n is the length of the string. Each binary search step takes O(n) time for computing rolling hashes. Space Complexity: O(n) for storing hash values in the hash set. Solution The solution uses binary search to decide on the candidate length L of the repeating substring. For each candidate length L, we use a rolling hash to compute hash values for all substrings of length L and store them in a hash set. If any hash is repeated, it confirms the presence of a repeating substring of length L. We try a longer length. Otherwise, we decrease the candidate length. The rolling hash is computed using a base (26, for lowercase letters) and a modulus (2^32 to limit integer overflow) which allows efficient hash recomputation for sliding windows. This approach efficiently narrows down the maximum repeating substring length. #153 Find the Index of the Largest Integer [Medium] Key Insights • Use binary search to efficiently narrow down the position of the unique large value. • At each step, divide the candidate range into two halves (or almost two halves when the range size is odd). Binary Search p.2257	Binary Search p.2258	Binary Search p.2259	Binary Search p.2260	Binary Search p.2261	Binary Search p.2256
Binary Search We use binary search over the possible values in the multiplication table, which range from 1 to m * n. For each midpoint i in the binary search, we compute the number of elements in the table that are less than or equal to x by iterating over each row and summing up min(x / i, n). This count tells us whether the left smallest element is to the left or right of x. We then adjust the search space boundaries accordingly. The approach leverages the multiplicative structure of the table and the monotonicity of the counting function. #710 Random Pick with Blacklist [Hard] Key Insights • The valid output range has a total count of available numbers W = n - len(blacklist). • Instead of generating random numbers and checking against a set, we can transform the range into a contiguous block [0, W - 1] and map any blacklist number within that block to a valid number in the upper range. • Mapping is precomputed for any blacklisted number x in [0, W - 1], assigns it a valid number from [W, n - 1] that is not blacklisted. • This approach ensures O(1) pick time after an O(n) precomputation where b = length of blacklist. Space & Time Time Complexity: constructor - O(b) for initializing data structures, pick() - O(1) per call. Space Complexity: O(b) for storing the blacklist set and mapping. Solution We compute the count of valid numbers, W = n - len(blacklist). For numbers in the blacklist that are less than W, we need to map them to a valid number from the range [W, n - 1]. To do this, we first store our blacklist numbers in the higher range [W, n - 1] in a set for quick lookup. Then for each blacklisted value in [0, W - 1], we iterate from W upwards to find a number that is not blacklisted and assign it to the blacklisted value. This mapping function, we generate a random integer r in [0, W - 1]. If r is in our mapping, we return its corresponding mapped value. Otherwise, we return r as it is already a valid number. #774 Minimize Max Sum to Gas Station [Hard] Key Insights • The optimal "penalty" (i.e., the maximum distance between adjacent stations after placement) can be found using binary search over a continuous distance range. • For each candidate distance (mid), simulate the number of gas stations required by iterating over each adjacent station gap and calculating: stations_{required} = ceil(gap / mid) - 1. Binary Search p.2263	Binary Search p.2264	Binary Search p.2265	Binary Search p.2266	Dynamic Programming #19 Dynamic Programming LeetCode - SimplifyLeet - WalICC - LeetCode Math - Dynamic Programming - Memoization Lists Grid 169 Time Complexity: O(n) Space Complexity: O(n) (can be optimized to O(1) using constant space) Problem You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top? Example 1: Input: n = 2 Output: 2 Explanation: There are two ways to climb to the top. 1 step + 1 step 2 steps Example 2: Input: n = 3 Output: 3 Explanation: There are three ways to climb to the top. 1 step + 1 step + 1 step 1 step + 2 steps 2 steps + 1 step Dynamic Programming p.2267	Dynamic Programming #70 Climbing Stairs LeetCode - SimplifyLeet - WalICC - LeetCode Math - Dynamic Programming - Memoization Lists Grid 169 Time Complexity: O(n) Space Complexity: O(n) (can be optimized to O(1) using constant space) Problem You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top? Example 1: Input: n = 2 Output: 2 Explanation: There are two ways to climb to the top. 1 step + 1 step 2 steps Example 2: Input: n = 3 Output: 3 Explanation: There are three ways to climb to the top. 1 step + 1 step + 1 step 1 step + 2 steps 2 steps + 1 step Dynamic Programming p.2268

Sheet 69/86 · mini-pages 2449-2494 · LeetMastery 36-up

Sheet 74/86 · mini-pages 2629-2664 · LeetMastery 36-up

<pre>def dfs(root, h): if root: if h == len(ans): ans.append(root.val) # pre-order, all the way to the right dfs(root.right, h + 1) dfs(root.left, h + 1) ans = [] dfs(root, 0) return ans</pre>	Trees & BST #230 Kth Smallest Element in a BST MED LeetCode · SimplifyLeet · WalkCC · LeetCode Tree · DFS · BST Lists: Grid 169 Time Complexity: O(n) in the worst-case scenario (when k = n), but on average the traversal may stop early. Space Complexity: O(h) where h is the height of the tree (O(n) in the worst-case for a skewed tree). Problem Given the root of a binary search tree, and an integer k, return the kth smallest value (1-indexed) of all the values of the nodes in the tree. Example 1:	<pre>Input: root = [3,1,4,null,2], k = 1 Output: 1</pre> Example 2:	<pre>Input: root = [5,3,6,2,4,null,null,1], k = 3 Output: 3</pre> Constraints: - The number of nodes in the tree is n. 1 ≤ k ≤ n ≤ 10⁴ 0 ≤ Node.val ≤ 10⁴ Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?	Community Solutions (Python) • WalkCC <pre>class Solution: def kthSmallest(self, root: TreeNode None, k: int) -> int: return dfs(countNodes(root: TreeNode None) -> int: if not root: return 0 return 1 + countNodes(root.left) + countNodes(root.right) leftCount = countNodes(root.left) if leftCount == k - 1: return root.val if leftCount >= k: return self.kthSmallest(root.left, k) return self.kthSmallest(root.right, k - 1 - leftCount) < leftCount < k</pre> • LeetCode	# Definition for a binary tree node. <pre>class TreeNode: def __init__(self, val=0, left=None, right=None): self.val = val self.left = left self.right = right</pre> class Solution: <pre>def kthSmallest(self, root: Optional[TreeNode], k: int) -> int: stk = [] while root or stk: if root: stk.append(root) root = root.left else: root = stk.pop() k -= 1 if k == 0: return root.val root = root.right</pre> • SimplifyLeet
Trees & BST p.2845	Trees & BST p.2846	Trees & BST p.2847	Trees & BST p.2848	Trees & BST p.2849	Trees & BST p.2850
<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val=0, left=None, right=None): self.val = val self.left = left self.right = right class Solution: def kthSmallest(self, root: TreeNode, k: int) -> int: # Initialize counter and result self.k = k self.ans = None # In-order traversal: left -> node -> right def inorder(node): if not node: return # Traverse left subtree inorder(node.left) # Process current node self.k -= 1 if self.k == 0: self.ans = node.val</pre>	<pre>self.k -= 1 if self.k == 0: self.ans = node.val return # Traverse right subtree if kth element not found inorder(node.right) inorder(root) return self.ans</pre> • LC.ca <pre># Definition for a binary tree node. class TreeNode: def __init__(self, val=0, left=None, right=None): self.val = val self.left = left self.right = right class Solution: def kthSmallest(self, root: Optional[TreeNode], k: int) -> int: stk = [] while root or stk: if root: stk.append(root) root = root.left else: root = stk.pop() self.k -= 1 if self.k == 0: return root.val root = root.right</pre>	<pre>if root: stk.append(root) root = root.left else: root = stk.pop() k -= 1 if k == 0: return root.val root = root.right ##### # Definition for a binary tree node. class TreeNode: def __init__(self, val=0, left=None, right=None): self.val = val self.left = left self.right = right class Solution: def kthSmallest(self, root: Optional[TreeNode], k: int) -> int: # Brute Force Approach class Solution: def lowestCommonAncestor(self, root, p, q): # Brute Force O(n) - find root-to-node paths, find last common node def path_to_node(node, target): path = [] while node: path.append(node) if target.val < node.val: node = node.left else: node = node.right return path</pre>	<pre>node = self.build(root) return self.dfs(node, k) class MyTreeNode: def __init__(self, x): self.val = x self.left = None self.right = None def build(self, root: TreeNode) -> MyTreeNode: if not root: return None node = self.MyTreeNode(root.val) node.left = self.build(root.left) node.right = self.build(root.right) if node.left: node.count += node.left.count if node.right: node.count += node.right.count</pre>	<pre>return node def dfs(self, node: MyTreeNode, k: int) -> int: if not node: cnt = node.left.count if k < cnt + 1: return self.dfs(node.left, k) elif k < cnt + 1: return self.dfs(node.right, k - 1 - cnt) else: return node.val else: if k == 1: return node.val return self.dfs(node.right, k - 1)</pre>	Trees & BST #235 Lowest Common Ancestor of a Binary Search Tree MED LeetCode · SimplifyLeet · WalkCC · LeetCode Tree · DFS · BST Lists: Grid 169 Time Complexity: O(h), where h is the height of the tree (O(log n) for balanced trees and O(n) in the worst-case skewed tree). Space Complexity: O(1) with an iterative approach (O(h) if using recursion). Problem Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST. According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself)". Example 1:
<pre>Input: root = [6,2,8,0,4,7,null,null,3,5], p = 2, q = 8 Output: 6 Explanation: The LCA of nodes 2 and 8 is 6.</pre> Example 2:	<pre>Input: root = [6,2,8,0,4,7,null,null,3,5], p = 2, q = 4 Output: 2 Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.</pre> Example 3: <pre>Input: root = [2,1], p = 2, q = 1 Output: 2</pre> Constraints:	• The number of nodes in the tree is in the range [2, 10⁵]. -10⁹ ≤ Node.val ≤ 10⁹ - All Node.val are unique. - p ≠ q p and q will exist in the BST. Brute Force (Python) <pre># Brute Force Approach class Solution: def lowestCommonAncestor(self, root, p, q): # Brute Force O(n) - find root-to-node paths, find last common node def path_to_node(node, target): path = [] while node: path.append(node) if target.val < node.val: node = node.left else: node = node.right return path</pre>	<pre>else: break return path p_path = path_to_root(p) q_path = path_to_root(q) p_set = set(p_path) for node in reversed(q_path): if node.val in p_set: return node Community Solutions (Python) • WalkCC class Solution: def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode',) -> 'TreeNode': if root.val < min(p.val, q.val): return self.lowestCommonAncestor(root.left, p, q) elif root.val < min(p.val, q.val): return self.lowestCommonAncestor(root.right, p, q) else: return root • SimplifyLeet • LeetCode</pre>	<pre># Definition for a binary tree node. class TreeNode: def __init__(self, val): self.val = val self.left = None self.right = None class Solution: def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode': while 1: if root.val < min(p.val, q.val): root = root.left elif root.val > max(p.val, q.val): root = root.right else: return root</pre> • SimplifyLeet	<pre>def lowestCommonAncestor(root, p, q): # Iterate until we find the lowest common ancestor while 1: # If both p and q are less than the current node, move to left subtree if p.val < root.val and q.val < root.val: root = root.left # If both are greater, move to right subtree elif p.val > root.val and q.val > root.val: root = root.right else: return root # Found the split point; return current node as LCA</pre> • LC.ca <pre># Definition for a binary tree node. class TreeNode: def __init__(self, x): self.val = x self.left = None self.right = None class Solution: def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode': if not root or root == p or root == q: return root left = self.lowestCommonAncestor(root.left, p, q) right = self.lowestCommonAncestor(root.right, p, q) if left and right: return left or right</pre>
<pre>) -> 'TreeNode': while 1: if root.val < min(p.val, q.val): </pre>					

Trees & BST	p.2947	Trees & BST	p.2948	Trees & BST	Sheet 82/86 · mini-pages 2017-2952 · sheet Mastery 36-up	p.2950	Trees & BST	p.2951	Trees & BST	p.2952
-------------	--------	-------------	--------	-------------	--	--------	-------------	--------	-------------	--------


```
Input: root = [5,6,1]
Output: 6.00000
Explanation:
For the node with value = 5 we have an average of  $(5 + 6 + 1) / 3 = 4$ .
For the node with value = 6 we have an average of  $6 / 1 = 6$ .
For the node with value = 1 we have an average of  $1 / 1 = 1$ .
So the answer is 6 which is the maximum.
```

Community Solutions (Python)

- WalkCC

Trees & BST

p.2996

```

graph TD
    1((1)) --> 3((3))
    1 --> 2((2))
    1 --> 4((4))
    3 --> 5((5))
    3 --> 6((6))
  
```

```

graph TD
    1((1)) --> 2((2))
    1 --> 3((3))
    1 --> 4((4))
    1 --> 5((5))
    3 --> 6((6))
    3 --> 7((7))
    7 --> 11((11))
    7 --> 15((15))
    5 --> 9((9))
    5 --> 10((10))
    9 --> 12((12))
    9 --> 13((13))
    10 --> 14((14))
    10 --> 16((16))
  
```

```

Brute Force (Python)

# Brute Force Approach
class Solution:
    def cloneTree(self, root):
        # Brute Force - collect inorder, then
        process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)

```

```

14:04:14
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not
None else []
14:04:14

class Solution:
    def cloneTree(self, root: 'Node') -> 'Node':

```

```
class Node {
public int val;
public List<Node> children;
}
```

```
Input: root = [1,mult,2,3,4,5,mult,mult,6,7,nell,8,mult,9,10,mult,11,mult,12,mult,13,mult,mult,14]
Output: [1,mult,2,3,4,5,mult,mult,6,7,nell,8,mult,9,10,mult,11,mult,12,mult,13,mult,mult,14]
```

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [0, 104].

Follow up: Can your solution work for the graph problem?

```

# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not
        None else []

def cloneTree(root):
    # Base case: if root is None, return None.
    if root is None:
        return None

    # Create a clone for the current node.
    cloned_node = Node(root.val)

    # Recursively clone each child and add it to the
    # cloned_node's children list
    for child in root.children:
        cloned_child = cloneTree(child)
        cloned_node.children.append(cloned_child)

    # Return the cloned node.
    return cloned_node

# Example usage:

```

```
def __clone__(self, val=None, children=None):
    self.val = val
    self.children = children if children is not
None else []
    return self

class Solution:
    def cloneTree(self, root: 'Node') -> 'Node':
        if root is None:
            return None
        children = [self.cloneTree(child) for
child in root.children]
        return Node(root.val, children)
```

LeetCode - SimplyLeet - WalkC - LeetCode
Tree - DFS
List: Premium 98
Time Complexity: $O(n)$, where n is the number of nodes, and each node is visited once. Space Complexity: $O(h)$, where h is the height of the tree, due to the recursion stack (worst-case $O(n)$ for a skew tree).

Problem
Given a root of an N -ary tree, you need to compute the length of the diameter of the tree.

The diameter of an N -ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.

(Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value.)

Example 1:

```
graph TD; 3((3)) --- 5((5)); 3 --- 6((6)); 2((2)) --- 3; 4((4)) --- 2; style 3 stroke:#f00; style 5 stroke:#f00;
```

Input: root = [1,mul,3,2,4,mul,5,6]
Output: 3
Explanation: Diameter is shown in red color.

Example 2:

```

graph TD
    2((2)) --- 3((3))
    2 --- 4((4))
    3 --- 5((5))
    4 --- 6((6))
  
```

Input: root = [1,null,2,null,3,4,null,5,null,6]
 Output: 4

Example 3:

```

graph TD
    2((2)) --- 3((3))
    2 --- 4((4))
    3 --- 6((6))
    3 --- 7((7))
    4 --- 8((8))
    4 --- 5((5))
    7 --- 11((11))
    8 --- 12((12))
    5 --- 9((9))
    9 --- 10((10))
    11 --- 14((14))
    style 2 stroke:#f00,stroke-width:2px
    style 3 stroke:#f00,stroke-width:2px
    style 7 stroke:#f00,stroke-width:2px
    style 11 stroke:#f00,stroke-width:2px
    style 14 stroke:#f00,stroke-width:2px
    linkStyle 0 stroke:#f00,stroke-width:2px
    linkStyle 1 stroke:#f00,stroke-width:2px
    linkStyle 4 stroke:#f00,stroke-width:2px
    linkStyle 5 stroke:#f00,stroke-width:2px
    linkStyle 8 stroke:#f00,stroke-width:2px
    linkStyle 9 stroke:#f00,stroke-width:2px
  
```

Input: root = [1,mult,2,3,4,5,mult,mult,6,7,mult,8,mult,9,10,mult,mult,11,mult,12,mult,13,mult,mult,14]
 Output: 7

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [1, 104].

```
class Solution:
    def diameter(self, root: 'Node') -> int:
        ans = 0

        def maxDepth(root: 'Node') -> int:
            """Returns the maximum depth of the subtree
            rooted at root."""
            nonlocal ans
            maxSubDepth1 = 0
            maxSubDepth2 = 0
            for child in root.children:
                maxSubDepth = maxDepth(child)
                if maxSubDepth > maxSubDepth1:
                    maxSubDepth2 = maxSubDepth1
                    maxSubDepth1 = maxSubDepth
                elif maxSubDepth > maxSubDepth2:
                    maxSubDepth2 = maxSubDepth
            ans = max(ans, maxSubDepth1 + maxSubDepth2)
            return 1 + maxSubDepth1

        maxDepth(root)

        return ans
```

```

class Node:
    # Definition for a Node.
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not
        None else []

class Solution:
    def diameter(self, root: 'Node') -> int:
        # :type root: 'Node'
        # :rtype: int

        def dfs(root):
            if root is None:
                return 0
            max_lans

```

```

n1 = n2 = 0
for child in root.children:
    t = dfs(child)
    if t > n1:
        n2, n1 = n1, t
    elif t >= n2:
        n2 = t
ans = max(ans, n1 + n2)
return n1 + n1

ans = 0
dfs(root)
return ans

```

• **Simplest**

• **Definition for a Node.**

```

class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None
        else []

```

class Solution:

```

def diameter(self, root: 'Node') -> int:

```

```
def dfs(node):
    if not node:
        return 0 # Base case: if the node
is None, return depth 0

    # Variables to hold the top two maximum
depths from the children
    max_depth1, max_depth2 = 0, 0

    # Iterate over each child to compute
their maximum depths.
    for child in node.children:

        child_depth = dfs(child) # Recurse
on the child

        # Update the two largest depths
if child_depth > max_depth1:
            max_depth2 = max_depth1
            max_depth1 = child_depth
        elif child_depth > max_depth2:
            max_depth2 = child_depth

    # Update the maximum diameter. The
candidate is the sum of the two largest
```

```
# Return the maximum depth from this
node.
    return max_depth + 1

dfs(root)
return self.max_diameter
```

```
# LC6
class Node:
    # Definition for a Node.
    class Node:
        def __init__(self, val=None, children=None):
            self.val = val
            self.children = children if children is not
            None else []

class Solution:
    def dfs(self, node: Node) -> int:
        if not node:
            return 0
        max_depth = 0
        for child in node.children:
            max_depth = max(max_depth, self.dfs(child))
        return max_depth + 1
```

```
def diameter(self, root: 'Node') -> int:
    """
    :type root: 'Node'
    :rtype: int
    """

    def dfs(root):
        if root is None:
            return 0
        nonlocal ans
        n1 = n2 = 0
        for child in root.children:
            t = dfs(child)
            if t > n1:
                n2, n1 = n1, t
            elif t >= n2:
                n2 = t
        ans = max(ans, n1 + n2)
        return 1 + n1

    ans = 0
    dfs(root)
    return ans
```

Trees & BST

1650 Lowest Common Ancestor of a Binary Tree

NEED

leetDeets : Simply Tree : WalkC# : LeetCode

Hash Table - Tree - Two Pointers

Tags : Trie : Binary Tree

Time : O(N) Space Complexity: O(1), where h is the height of the tree.

Problem

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).

Each node will have a reference to its parent node. The definition for Node is below:

```

class Node {
public int val;
public Node left;
public Node right;
public Node parent;
}
```

According to the definition of LCA on Wikipedia: "The lowest common ancestor of two nodes p and q in a tree is the lowest node that has both p and q as descendants (where we allow a node to be a descendant

Example 1:

```
graph TD; 3((3)) --> 5((5)); 3 --> 1((1)); 5 --> 6((6)); 5 --> 2((2)); 1 --> 0((0)); 1 --> 8((8)); 2 --> 7((7)); 2 --> 4((4));
```

Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 8
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:

```

graph TD
    3((3)) --- 5((5))
    3 --- 1((1))
    5 --- 6((6))
    5 --- 2((2))
    1 --- 0((0))
    1 --- 8((8))
    2 --- 7((7))
    2 --- 4((4))
  
```

Input: root = [3,5,1,6,2,8,8,m1,m1,7,4], p = 5, q = 4
 Output: 5
 Explanation: The LCA of nodes 5 and 4 is 5 since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2
 Output: 1

```
# The number of nodes in the tree is in the range [2, 100]
- 109 <= Node.val < 109
- All Node.val are unique.
- p = q
- p and q exist in the tree.

Brute Force (Python)

# Brute Force Approach
class Solution:
    def lowestCommonAncestor(self, p, q):
        # Brute force - collect inorder, then
        process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)
        collect(p)
        return vals[0] if vals else 0

Community Solutions (Python)

• WalCC
```

```
class Solution:
    # Same as 160. Intersection of Two Linked Lists
    def lowestCommonAncestor(self, p: 'Node', q:
'Node') -> 'Node':
    a = p
    b = q

    while a != b:
        a = a.parent if a else q
        b = b.parent if b else p

    return a

# LeetCode
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
```

```
class Solution:
    def lowestCommonAncestor(self, p: "Node", q:
"Node") -> "Node":
        vis = set()
        node = p
        while node:
            vis.add(node)
            node = node.parent
        node = q
        while node not in vis:
            node = node.parent
        return node
```

```
# Simplexlet
# Definition for a Node.
# class Node:
#     def __init__(self, val, left=None, right=None,
parent=None):
#         self.val = val
#         self.left = left
#         self.right = right
#         self.parent = parent

def lowestCommonAncestor(p, q):
    # Compute the depth for node p
```


